

Implementing Encryption Infrastructure Management and Redevelopment in a Post-Quantum World: An Exploration of Nascent Quantum-Resistant Encryption Algorithms.

Zachary Bradley

The University of North Georgia

CYBR 4950: Cybersecurity Capstone

January 28, 2025

Abstract

This research project explores various implementations of post-quantum cryptography, infrastructure security, and currently available offensive quantum computing strategies via the IBM Quantum Platform, the Qiskit Software Development Kit, the Open Quantum Safe Liboqs C Library for post-quantum cryptographic algorithms, and a Kali Linux Virtual Machine. In conjunction with general topical research on quantum information systems, offensive quantum computing, and post-quantum cryptographic algorithms, a series of technical executions have taken place on the Qiskit Aer quantum simulator and IBM QPU hardware units. These explorations have implemented an IBM guided “Quantum Hello World”, an independent toy execution of Shor’s algorithm on the Aer simulator, and an execution of the Deutsch-Jozsa algorithm on an IBM QPU. From a defensive perspective, benchmark testing of the National Institute of Standards and Technologies FIPS 203, 204, and 205 post-quantum cryptographic algorithms and protocol suites have been executed on a Kali Linux virtual machine for the purposes of practical implementation research. Associated infrastructure recommendations have been made in addition to more in-depth research regarding the topic of cryptographic security in a post-quantum infrastructural environment.

Implementing Encryption Infrastructure Management and Redevelopment in a Post-Quantum World: An Exploration of Nascent Quantum-Resistant Encryption Algorithms.

Our contemporary information technology infrastructure is currently in the midst of extreme turbidity. With the widespread personal usage of Large Language Models and Generative AI, the continued advancement of microprocessor speed and size marching in lockstep with Moore’s Law (Machovech, 2023), and the ever-pervasive social encroachment presented by social media developers steadily influencing our mobile device usage in novel ways, it is difficult to track the vulnerabilities that are generated by these novel technologies at a pace sufficient to protect the integrity of both personal and enterprise security. In conjunction with these advancements and technological plot twists, a major infrastructure overall is desperately needed in the transition from IPv4 to IPv6, which, at completion, is conservatively projected to cost in excess of \$73 billion dollars over 20 years to U.S stakeholder groups alone (Gallagher, 2005, p. 42).

Arriving parallel with this constant field of ground-breaking infrastructural disruptions, an alternate universe of paradigm shifting computational power is looming over the near horizon, quantum computing. While estimates vary, the general contemporary consensus projects industrially actionable quantum computing within five to fifteen years (Barrett, 2025). With the arrival of these novel quantum computational approaches, problems considered functionally impossible to solve by the most affective contemporary classical computers can, and will be, solved by quantum computers. This incredibly dramatic increase in computational power, combined with the utilization of Shor’s algorithm, will be able to crack many of our

contemporary information technology infrastructure's most relied-upon encryption suites (National Institute of Standards and Technology, 2024). For example, the RSA public-key cryptosystem, the elliptic-curve Diffie-Helman key exchange, and the finite field Diffie-Helman key exchange will all be rendered useless by the usage of quantum computing mechanisms. This projected disruption of the encryption protocols that underlie the secure functionality of our contemporary information infrastructure will have a profound impact on the methods with which we use to secure our most valuable information transfers.

In anticipation of the arrival of actionable quantum computing, the National Institute of Standards and Technology (NIST), initiated a search and evaluation process for novel quantum-resistant public-key cryptographic algorithms in 2016. On August 13, 2024, NIST announced the three finalized post-quantum encryption standards, FIPS 203, a standard that is based on the CRYSTALS-Kyber algorithm (now renamed ML-KEM, or the Module-Lattice-Based Key Encapsulation Mechanism), FIPS 204, an algorithm aimed at securing digital signatures using the CRYSTALS-Dilithium algorithm (now renamed ML-DSA, or the Module-Lattice-Based Digital Signature Algorithm), and FIPS 205, an algorithm that utilizes the Sphincs+ algorithm to provide continued support for digital signatures (now renamed SLH-DSA, or the Stateless Hash-Based Digital Signature Algorithm). These algorithms, again utilizing the Module-Lattice Based Key Encapsulation Mechanism, the Module-Lattice based Digital Signature Algorithm and the Stateless Hash-Based Signature schemes, are projected to remain impenetrable under the gaze of a future quantum computer and Shor's algorithm.

The scope of this project is to evaluate, explore, and challenge the affective deployment of these algorithms in the commercial, theoretical and technical space. My goal is to conduct research on all three of the NIST supported Post-Quantum Encryption Algorithms with regards to the impact these new computational encryption suites will have on the forthcoming necessary shift in software, hardware, and encryption suite management services across enterprise level usage deployment factors. NIST has made a call for IT professionals to evaluate the current cryptographic infrastructure and begin the process of preparing for the inevitable shift to these Post-Quantum Encryption Standards; I will employ this request throughout the course of this research project.

While there are many disrupting factors at play, this upcoming shift in technological infrastructure will mirror the depth and expense of the aforementioned shift from IPv4 to IPv6. Both transitions require gradual and sustained changes across many infrastructural zones and will only be completed through the injection of significant capital overhead. The economic and policy factors involved in this transition to a post-quantum environment will be evaluated within the context of a projected enterprise model.

Unfortunately, access to a quantum computer is not possible at this time, so for my technical exploration I will be pursuing my research of both general quantum computing encryption principles and the NIST Post-Quantum Encryption Protocols with the IBM Quantum

Platform and Qiskit quantum software development kit. Other general encryption requirements involving the FIPS 203, 204, and 205 algorithms will take place utilizing the Open Quantum Safe Liboqs C Library for post-quantum cryptography on my personal device within a Kali Linux virtual machine or in the Cyber Lab at UNG.

Contextualizing Quantum Resistant Encryption and Potential Threats

- A Quantum Computing Primer

Quantum Information Processing and the Qubit:

Quantum computing and the physical principles associated with its current functionality operate on some of the furthest horizons of human understanding. Within the field of quantum physics, and contained inside of that nebulous space, quantum mechanics, the guiding principles of reality depart markedly, and the processes with which these concepts operate manifest themselves in counterintuitive and bewildering ways. To quote Werner Heisenberg, “Not only is the Universe stranger than we think, it is stranger than we can think” (Amerongen, 2021).

While it may seem hyperbolic at first, it is appropriate to describe quantum computing devices as a form of computational technologies which leverage the paradoxical foundations of reality to process information at a speed and scope that is simply unattainable for classical computers operating over the course of thousands, millions, if not billions of years of computation time (NIST, 2024).

A broad definition of quantum computing, according to a Congressional Research Service publication on the subject, states “Quantum computing, one of technology applications of quantum information science, uses a quantum bit, or qubit, as its basic data unit, to harness quantum properties such as superposition and entanglement. By generating and manipulating qubits, a quantum computer is capable of performing certain calculations significantly faster than conventional, non-quantum computers, known as classical computers, leading to new ways to solve some complex problems that were previously unsolvable (Zhu, 2023).”

While there are a number of different methodologies involved in the creation, capture, and manipulation of qubits, a select few tested methods have delivered the most consistent results throughout the genesis of this new computing paradigm. According to Microsoft’s online Azure Quantum documentation, quantum qubit creation, capture, and manipulation can occur through the utilization of photonic qubits, superconducting qubits, quantum dot qubits, diamond nitrogen-vacancy center qubits, and topological qubits, among others (Microsoft, n.d). This plethora of qubit development options presents a diversity of mechanical pathways for the construction of quantum computing and quantum information system devices.

Quantum Information Processing and the Qubit: The Apple Orchard Metaphor

The density and complexity of quantum computing cannot be understated; the physical model upon which its foundational functional principles are based can be counterintuitive,

surreal, and bewildering. A common approach to distilling complex concepts into more digestible content is to develop a metaphor to assist in our baseline understanding. For the purposes of comparing classical and quantum computing, I suggest that we consider this metaphorical explanation; The scope of computational principles and the data sets upon which they operate can be allegorically described as The Apple Orchard (the raw data) and The Harvesters (the computational principles which process the data i.e. classical computers and their associated information systems vs quantum computers and their associated information systems).

In the instance of classical computers operating on data sets, the allegorical comparison would be described as a singular harvester working in The Orchard to harvest apples. The harvester, working by himself, collects as many apples as he can as he works his way down the rows of trees through the workday. Over time, he has become stronger, faster, and more agile as he picks the apples and places them in his basket, but despite his best efforts, he can only move so quickly, and harvest from so many apple trees on a daily basis.

One day, The Harvester wakes up feeling strange; he enters the orchard and suddenly experiences a multiplication. He now finds himself existing across the entire apple orchard. At this point, there are thousands of the same Harvester, standing in front of every single apple, on every single branch, on every single tree. In one fluid motion, he and his multitudes pick every single apple in the orchard and place in in their pocket.

The contrast in experiences between the Harvester of yesterday, and the Harvester of tomorrow, is the difference between classical and quantum computing.

Contextualizing Quantum Resistant Encryption and Potential Threats

- A Quantum Computing Primer

Quantum Information Processing in Practice: An Introduction to Qiskit

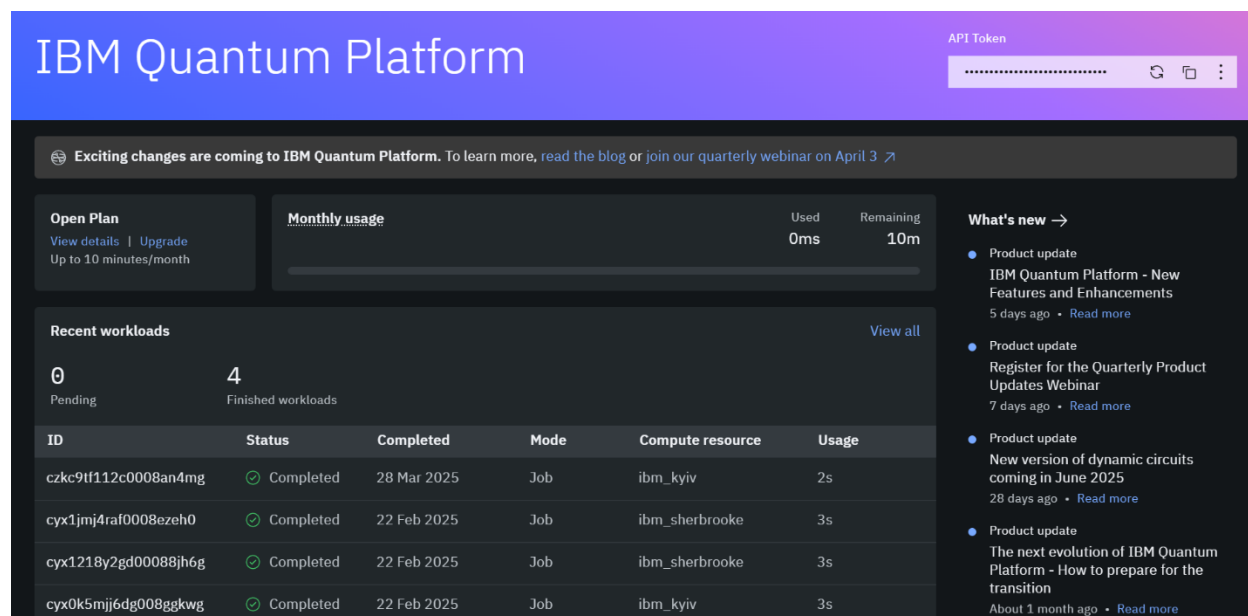
For an introduction to the concepts of quantum computing in practice, we will turn to IBM's Quantum Platform and the Qiskit Software Development Kit (SDK). Qiskit itself is an open-source software development platform which functions to support the execution of programs on quantum computers. Some examples of quantum program implementations would be operators, primitives, and extended quantum circuits (static, dynamic, and scheduled.) The IBM Quantum Platform notes the functionality of certain Qiskit features which include, a circuit building tool with (`qiskit.circuit`), a circuit library (`qiskit.circuit.library`), a quantum information library utilizing (`qiskit.quantum_info`), a transpiler for executing on QPU's (`qiskit.transpiler`), and (`qiskit.primitives`), a module which provides a primitive base for various hardware implementations.

In addition to Qiskit, real-time interaction with IBM's Quantum Processing Units (QPU's) requires the installation of Qiskit runtime (IBM Quantum Documentation, 2024) and additional supporting libraries. Qiskit Runtime functions as a cloud-based service for executing

quantum programs with error mitigation and suppression folded into hybrid computation approach; Qiskit utilizes both classical and quantum processors to reduce the errors and improve the quality of the results that are delivered by IBM's QPU's in practice.

To begin, I created an account on IBM's Quantum Platform, generated an IBMid with my University of North Georgia email and received an API token to begin interfacing with the system. This "open plan" allows me ten minutes per month of interaction with 3 IBM QPU's. My selection includes IBM_sherbrooke, IBM_kiv, and IBM_brisbane.

My dashboard is presented in the image below.

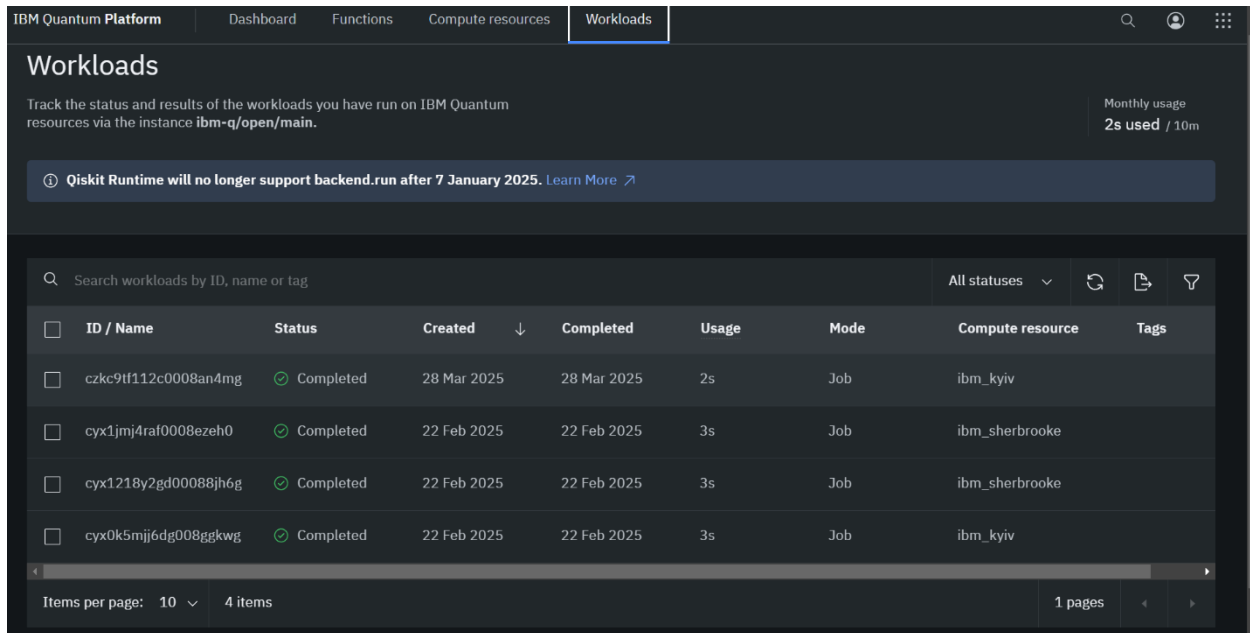


My API Token can be regenerated, and the center of the dashboard reveals the previous workloads, their ID numbers, and on which IBM QPU they were processed.

After I initialized my IBM Quantum Platform account and retrieved my API Token, I created a directory for the Qiskit SDK, created a python virtual environment (venv) within which to host the associated libraries, and installed Jupyter for code visualization support within the environment (IBM, 2025). Following initialization of Qiskit, I authenticated my account and connected to the IBM Cloud. I also made the decision to save my credentials locally for ease of use and a more seamless workflow environment while interfacing with the IBM Quantum Platform (IBM, 2025).

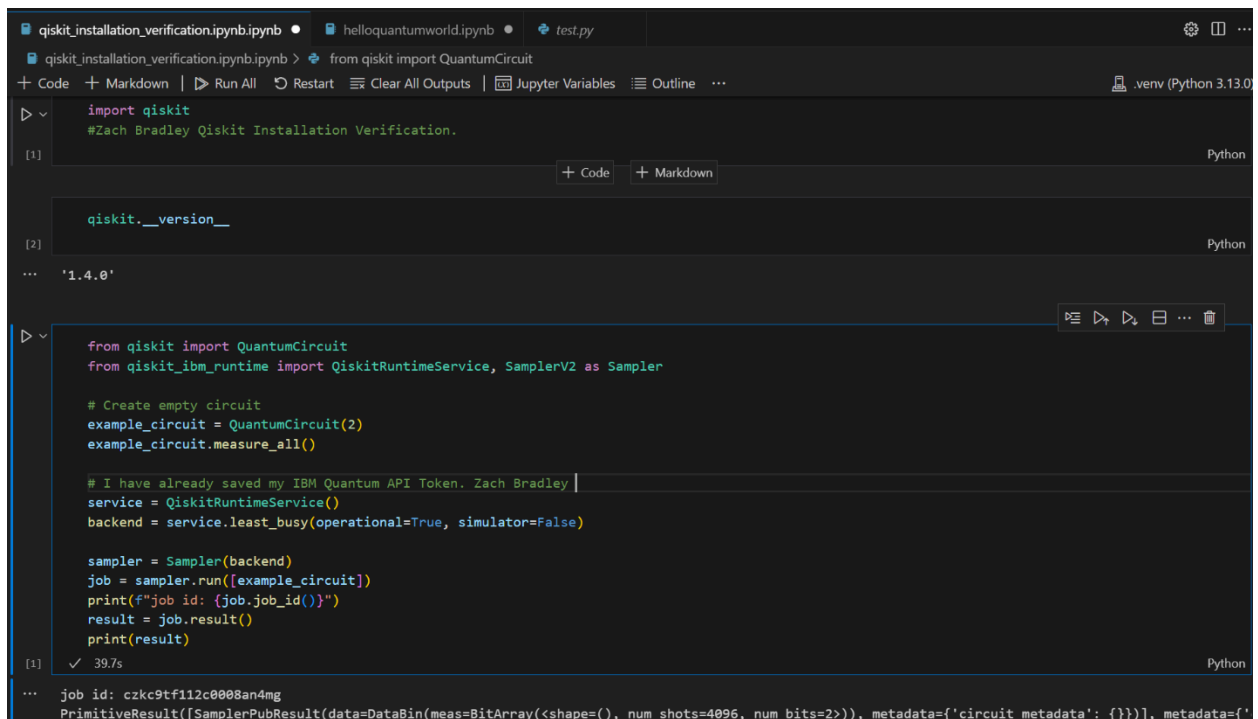
While the IBM Quantum platform offers many different approaches and development suites for simulating quantum computation locally or on the cloud, I have opted to conduct this project using IBM QPU instantiations.

Here is an example of some of the initial workloads I completed during setup and testing of my environment



IBM Quantum Platform								
Workloads								
Track the status and results of the workloads you have run on IBM Quantum resources via the instance <code>ibm-q/open/main</code> .								Monthly usage 2s used / 10m
Qiskit Runtime will no longer support <code>backend.run</code> after 7 January 2025. Learn More								
Search workloads by ID, name or tag								
All statuses								
ID / Name	Status	Created	Completed	Usage	Mode	Compute resource	Tags	
czkc9tf112c0008an4mg	Completed	28 Mar 2025	28 Mar 2025	2s	Job	ibm_kyiv		
cyx1jmi4ra10008ezeh0	Completed	22 Feb 2025	22 Feb 2025	3s	Job	ibm_sherbrooke		
cyx1218y2gd00088jh6g	Completed	22 Feb 2025	22 Feb 2025	3s	Job	ibm_sherbrooke		
cyx0k5mjj6dg008ggkwg	Completed	22 Feb 2025	22 Feb 2025	3s	Job	ibm_kyiv		
Items per page: 10 4 items 1 pages								

The IBM Quantum Platform Initialization Verification



```
qiskit_installation_verification.ipynb.ipynb • helloquantumworld.ipynb • test.py
qiskit_installation_verification.ipynb.ipynb > from qiskit import QuantumCircuit
+ Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
Python 3.13.0

[1] import qiskit
    #Zach Bradley Qiskit Installation Verification.

[2] qiskit.__version__
... '1.4.0'

[1] from qiskit import QuantumCircuit
    from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

    # Create empty circuit
    example_circuit = QuantumCircuit(2)
    example_circuit.measure_all()

    # I have already saved my IBM Quantum API Token. Zach Bradley
    service = QiskitRuntimeService()
    backend = service.least_busy(operational=True, simulator=False)

    sampler = Sampler(backend)
    job = sampler.run([example_circuit])
    print(f"job id: {job.job_id()}")
    result = job.result()
    print(result)
[1] ✓ 39.7s
... job id: czkc9tf112c0008an4mg
    PrimitiveResult([SamplerPubResult(data=DataBin(meas=BitArray(<shape=(), num_shots=4096, num_bits=2>)), metadata={'circuit_metadata': {}})], metadata={'
```

This initialization took a little bit of time, 39.7 seconds, as it was completed on the IBM_kyiv QPU.

For the purposes of this project, I have decided to begin my quantum computing exploration in accordance with the IBM Quantum Platforms guidance and replicate the quantum “Hello World” program. For the visualization aspect of this program, I have also imported the matplotlib package for visualization within the Jupyter notebooks environment. Executing this “Hello Quantum World” program within the IBM Quantum Platform will serve as a baseline and “proof of concept” for our introductory foray into the world of quantum computing.

The code implemented in the following cells is provided by the IBM Quantum environment; any alterations are my own and were applied to solve deprecation issues. The code cells are run locally on my own machine, where I have set up an IBMid account, a python virtual environment to envelop the Qiskit library, a Jupyter notebook and an IBM QPU channel to interface directly with actual quantum hardware,

Hello Quantum World

Exploring Quantum Circuits with IBM Qiskit:

To begin our rudimentary exploration of quantum computing we will run a code cell in Qiskit which initializes a two-qubit Bell state. IBM defines a Bell State as “a state wherein two qubits are fully entangled with each other (IBM, 2025).” The concept of particle entanglement, and in particular, the operation of Bell States discovered and detailed by John Stewart Bell, are a fundamental concept of quantum physics and likewise are a foundational operational principle of quantum computing. Two or more quantum particles can become entangled when any alteration of their individual inherent quantum states directly correlates to a change in the state of the associated particle (Bell, 1964). In other words, there is no independent operation that can be performed on the quantum state of entangled particles regardless of distance or time.

The entanglement of these particles (electrons or photons with correlative entanglement involving the electron “spin” or photon polarity) can occur in tandem or in opposition to one another. In the case of anti-correlation one electron may “spin” up, while its associated, entangled partner will instantaneously spin down or a photon’s polarity may be positively correlated in an instance where a photon with a left circular polarization is entangled with an associated photon which manifests left circular polarization as well. For the purpose of the photon example, this polarization can also occur with horizontal/vertical linear polarization properties of the photon. These states of entanglement are vital to the concept of logical operation within the operations of a quantum computer, the more entangled bits (qubits) that are available, the more computing power is available.

Bell States (Maximally Entangled Two-Qubit States)

1. Symmetric States:

- $|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$

- $|\Phi^-\rangle = \frac{1}{\sqrt{2}} (|00\rangle - |11\rangle)$

2. Antisymmetric States:

- $|\Psi^+\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |10\rangle)$

- $|\Psi^-\rangle = \frac{1}{\sqrt{2}} (|01\rangle - |10\rangle)$

This graphic explains the four states Bell states in modern quantum mathematical notation. It was generated using the DeepSeek Large Language Model (Deepseek, 2024), which in turn based its replication of the Bell state functions on data retrieved from Clauser, J. F., Horne, M. A., Shimony, A., & Holt, R. A. (1969). Proposed Experiment to Test Local Hidden-Variable Theories. *Physical Review Letters*, 23(15), 880–884. <https://doi.org/10.1103/PhysRevLett.23.880>

The four Bell states expressed in Python/Qiskit.

```
helloquantumworld.ipynb > import numpy as np
+ Code + Markdown | ▶ Run All ↺ Restart ☰ Clear All Outputs | (xv) Jupyter Variables ☰ Outline ...

▶ ~
import numpy as np
from qiskit.quantum_info import Statevector

# Define all four Bell States Zach Bradley.
def create_bell_state(state_type):
    if state_type == 'phi_plus':
        sv = Statevector.from_label('00') + Statevector.from_label('11')
    elif state_type == 'phi_minus':
        sv = Statevector.from_label('00') - Statevector.from_label('11')
    elif state_type == 'psi_plus':
        sv = Statevector.from_label('01') + Statevector.from_label('10')
    elif state_type == 'psi_minus':
        sv = Statevector.from_label('01') - Statevector.from_label('10')
    else:
        raise ValueError("Invalid Bell state type")
    return sv / np.sqrt(2) # Normalize

# Example usage:
bell = create_bell_state('phi_plus')
print("|0+> =", bell)

[4] ✓ 0.0s

... |0+> = Statevector([0.70710678+0.j, 0. +0.j, 0. +0.j,
                    0.70710678+0.j],
                    dims=(2, 2))
```

In further describing the context within which quantum programs operate, The IBM Quantum Platform Hello World guide describes the environment of quantum programs in this manner, “In a quantum program, *quantum circuits* are the native format in which to represent quantum instructions, and *operators* represent the observables to be measured.” (IBM, 2025). The genesis of our Hello Quantum World program begins with the development of a simple quantum circuit that creates a two- qubit Bell state. In the following Qiskit code cell, a simple quantum circuit is created with `qc = QuantumCircuit(2)`. A Hadamard gate transforms the qubit into a state of superposition. In this case, the Hadamard gate turns the 0 qubit into a probabilistic equivalent

of both states 0 and 1. This example can be further expressed using some simple mathematical expressions: The Hadamard gate Transforms $|0\rangle$ into an equal superposition of $|0\rangle$ and $|1\rangle$ ($|+\rangle$).

Once the Hadamard gate has been applied to qubit 0, a controlled-X (CNOT) gate is performed on qubit 1 which is operated or controlled by qubit 0. The controlled-X is the quantum equivalent of the XOR logical operator, and the combination of a Hadamard gate being applied to a control qubit (qubit 0) and a target qubit with a CNOT (qubit 1) creates a state of particle entanglement.

We will execute Bell state code cells using the Aer quantum simulator, for proof of concept, before we transition to running code on the IBM QPU.

Using the Aer simulator, we confirm the entanglement state of the simulated qubits. The results are 50% 50% for 00 and 11.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator # Correct import for Qiskit 1.0+
from qiskit.visualization import plot_histogram
#Zach Bradley Post Quantum Encryption Protocols
# Create Bell State  $|0^+\rangle$ 
def create_bell_circuit():
    qc = QuantumCircuit(2, 2)
    qc.h(0)
    qc.cx(0, 1)
    qc.measure_all() # Simpler measurement
    return qc

# Run simulation
simulator = AerSimulator()
bell_circuit = create_bell_circuit()
result = simulator.run(bell_circuit, shots=1000).result()
counts = result.get_counts()

print("Measurement results:")
print(counts) # Should show ~50% '00' and ~50% '11'
plot_histogram(counts)
```

[13] ✓ 0.9s

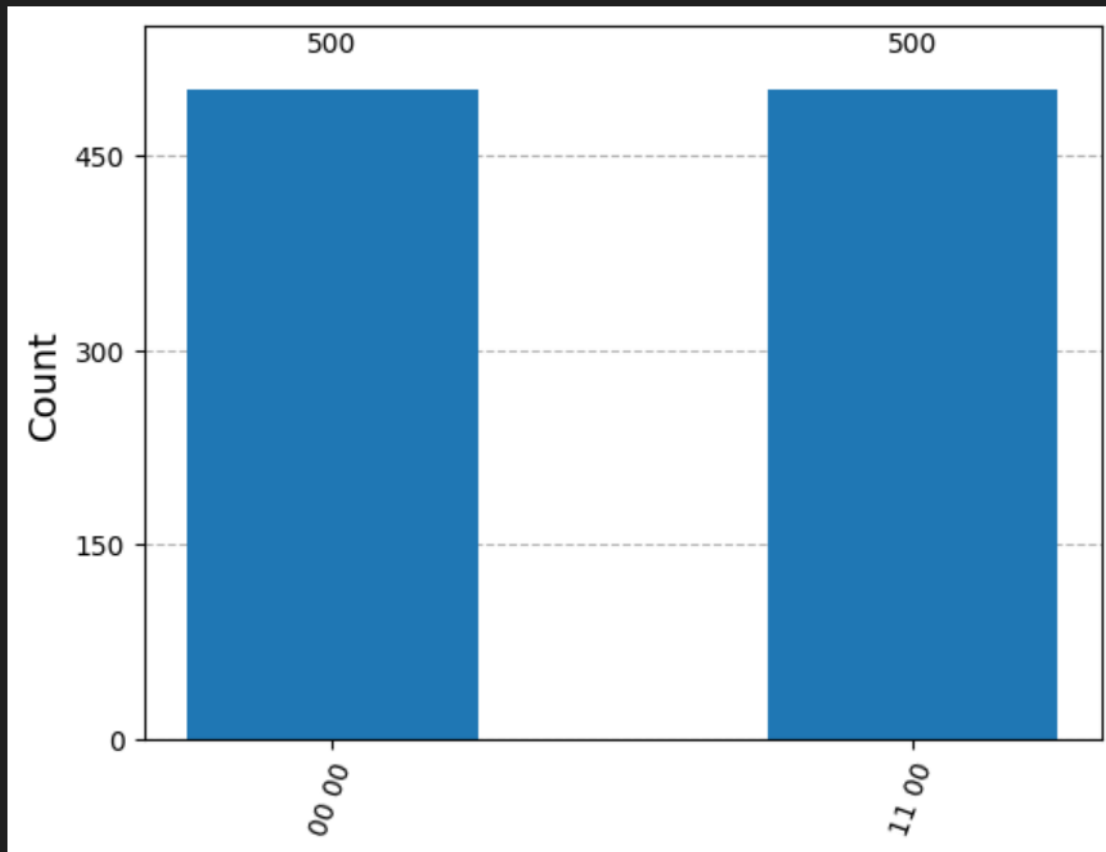
```
... Measurement results:
{'00 00': 500, '11 00': 500}
```

This demonstrates perfect probabilistic results since it was run on the Aer simulator, if we were to run this on an IBM QPU, it would likely be less than 50% due to natural decoherence. This issue of decoherence is one of the predominant roadblocks in place that has slowed the development of utility-grade quantum computing, error correction, reduction, and suppression has proven to be very difficult, and will continue to be both a dampening effect on the efficacy of

future quantum computers, and a present vulnerability for any forthcoming quantum internet infrastructure (Emerging Technology from the arXiv, 2024).

The plotted histogram visually demonstrates the accuracy of the simulated results. It features no decoherence in the measurement results as the simulated qubits are perfectly entangled.

```
Measurement results:  
{'00 00': 500, '11 00': 500}
```



All four Bell states run on the AerSimulator.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
#Zach Bradley Post Quantum Encryption Protocols.

def get_bell_state(state_type):
    qc = QuantumCircuit(2)
    if state_type == 'phi_plus': #  $(|00\rangle + |11\rangle)/\sqrt{2}$ 
        qc.h(0); qc.cx(0, 1)
    elif state_type == 'phi_minus': #  $(|00\rangle - |11\rangle)/\sqrt{2}$ 
        qc.h(0); qc.cx(0, 1); qc.z(1)
    elif state_type == 'psi_plus': #  $(|01\rangle + |10\rangle)/\sqrt{2}$ 
        qc.h(0); qc.x(1); qc.cx(0, 1)
    elif state_type == 'psi_minus': #  $(|01\rangle - |10\rangle)/\sqrt{2}$ 
        qc.h(0); qc.x(1); qc.cx(0, 1); qc.z(1)
    qc.measure_all()
    return qc

# Test all states
simulator = AerSimulator()
for state in ['phi_plus', 'phi_minus', 'psi_plus', 'psi_minus']:
    counts = simulator.run(get_bell_state(state), shots=1000).result().get_counts()
    print(f"{state}: {counts}")
```

[14] ✓ 0.0s

```
... phi_plus: {'00': 506, '11': 494}
    phi_minus: {'00': 518, '11': 482}
    psi_plus: {'01': 494, '10': 506}
    psi_minus: {'01': 513, '10': 487}
```

Before initiating the code for Hello Quantum World on the IBM QPU's we will create a two-qubit Bell state with a Hadamard gate applied to qubit 0 and a controlled-X (CNOT) gate on qubit 1 utilizing the code from IBM's Quantum Platform and plotting it with Matplotlib in the virtual environment.

```
#This will involve the creation of a two-qubit Bell state for initiation purposes.
#Zach Bradley Post-Quantum Encryption Protocols.
#This code is from https://docs.quantum.ibm.com/guides/hello-world.
from qiskit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import EstimatorV2 as Estimator

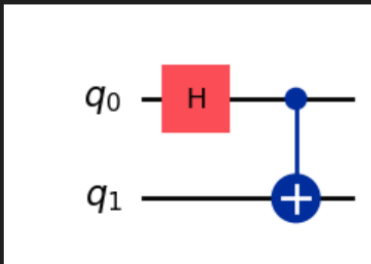
# Create a new circuit with two qubits
qc = QuantumCircuit(2)

# Add a Hadamard gate to qubit 0
qc.h(0)

# Perform a controlled-X gate on qubit 1, controlled by qubit 0
qc.cx(0, 1)

# Return a drawing of the circuit using Matplotlib ("mpl"). This is the
# last line of the cell, so the drawing appears in the cell output.
# Remove the "mpl" argument to get a text drawing.
qc.draw("mpl")
```

✓ 5.1s



This simple circuit setup reveals simulated entanglement between two qubits, In the next step, we will utilize the IBM QPU for actual quantum computing.

In this step, we will generate a quantum circuit and optimize its functionality for the ISA (Instruction Set Architecture) so it performs correctly and within range on the IBM QPU.

```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService()
#Zach Bradley Post-Quantum Encryption Protocols.
backend = service.least_busy(simulator=False, operational=True)

# Convert to an ISA circuit and layout-mapped observables.
pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
isa_circuit = pm.run(qc)

isa_circuit.draw("mpl", idle_wires=False)
```

✓ 14.8s

Global Phase: $7\pi/4$



Now we will execute the quantum primitives using the estimator class to map an estimated value of the observable. This sample set is gathered by running a circuit set a number of times in order to provide more accurate results. This sample set is run using a PUB or primitive unified block (IBM, 2025).

```
# Construct the Estimator instance.
# Zach Bradley Post Quantum Encryption Protocols.
# I had to set up six two-qubit Pauli Operators |

observables_labels = ["IZ", "IX", "ZI", "XI", "ZZ", "XX"]
observables = [SparsePauliOp(label) for label in observables_labels]

estimator = Estimator(mode=backend)
estimator.options.resilience_level = 1
estimator.options.default_shots = 5000

mapped_observables = [
    observable.apply_layout(isa_circuit.layout) for observable in observables
]

# One pub, with one circuit to run against five different observables.
job = estimator.run([(isa_circuit, mapped_observables)])

# Use the job ID to retrieve your job data later
print(f">>> Job ID: {job.job_id()}")
```

✓ 6.4s

>>> Job ID: czpztfnhqag008s94d0

Here we will plot the results of the primitive unified block of the six two-qubit Pauli Operators.

```
#Zach Bradley Post-Quantum Encryption Protocols
job_result = job.result()

pub_result = job_result[0]

# Plot the result
#Zach Bradley Post-Quantum Encryption Algorithms
from matplotlib import pyplot as plt

values = pub_result.data.evs

errors = pub_result.data.stds

# plotting graph
plt.plot(observables_labels, values, "-o")
plt.xlabel("Observables")
plt.ylabel("Values")
plt.show()
```

✓ 1.9s

✓ 0.1s

This code cell creates a quantum circuit for a 100-qubit GHZ state.

Setting up an array for the Pauli operators.

[illegible]

Processing the code circuits to match the backend server's ISA.

```
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService

#Zach Bradley Post-Quantum Encryption Algorithms.

service = QiskitRuntimeService()

backend = service.least_busy(
    simulator=False, operational=True, min_num_qubits=100
)
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)

isa_circuit = pm.run(qc)
isa_operators_list = [op.apply_layout(isa_circuit.layout) for op in operators]
```

✓ 14.2s

This action took 14.2 seconds since we are running this on IBM QPU hardware versus a simulator.

Now we will create an estimator object and send our request to the IBM QPU hardware.

```
from qiskit_ibm_runtime import EstimatorOptions
from qiskit_ibm_runtime import EstimatorV2 as Estimator

#Zach Bradley Post-Quantum Encryption Protocols.

options = EstimatorOptions()
options.resilience_level = 1
options.dynamical_decoupling.enable = True
options.dynamical_decoupling.sequence_type = "XY4"

# Create an Estimator object
estimator = Estimator(backend, options=options)
```

✓ 0.0s

```
# Submit the circuit to Estimator
#Zach Bradley Post-Quantum Encryption Protocols.
job = estimator.run([(isa_circuit, isa_operators_list)])
job_id = job.job_id()
print(job_id)
```

✓ 3.0s

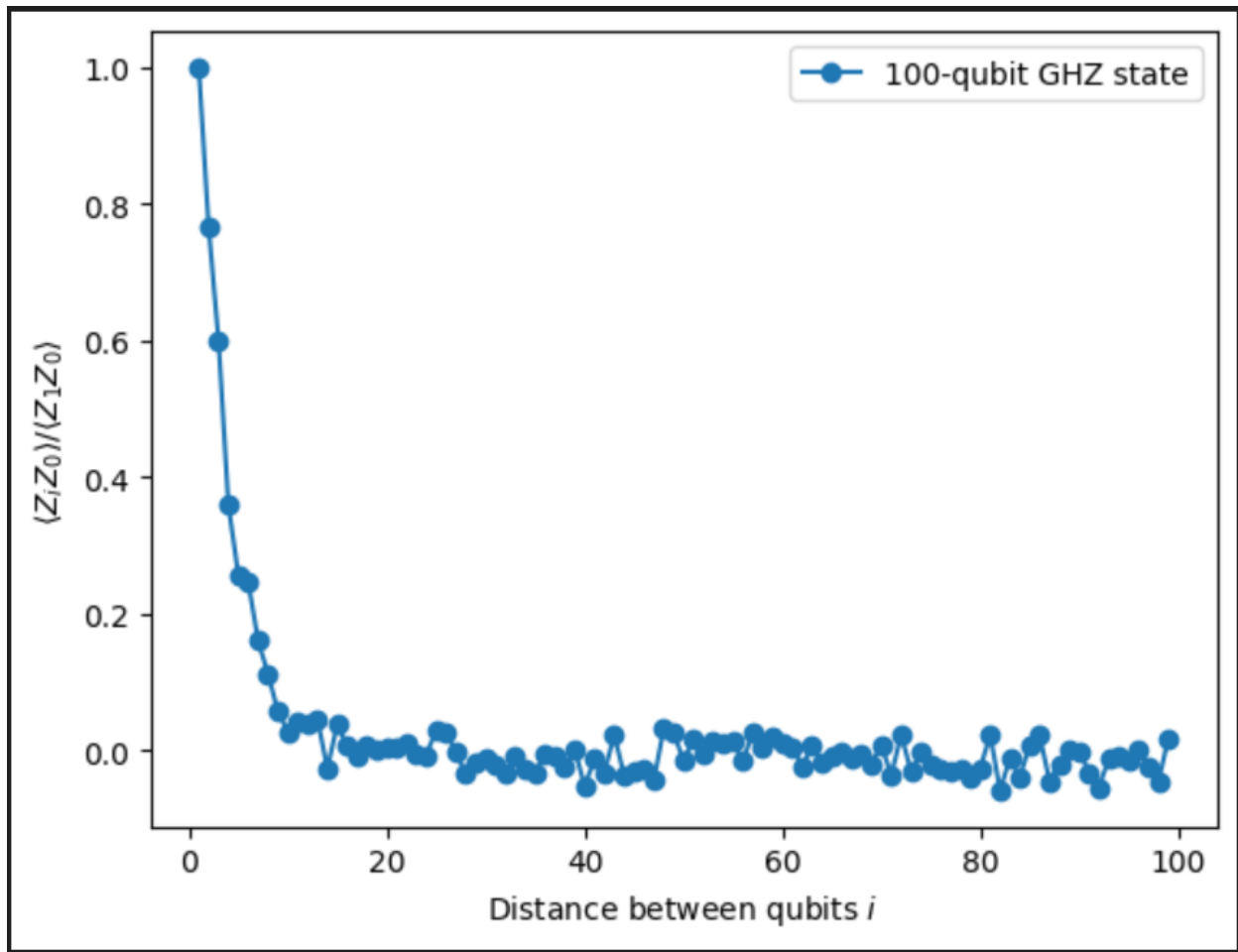
Processing the results delivered to us by the IBM QPU.

```
import matplotlib.pyplot as plt
from qiskit_ibm_runtime import QiskitRuntimeService

# data
data = list(range(1, len(operators) + 1)) # Distance between the Z operators
result = job.result()[0]
values = result.data.evs # Expectation value at each Z operator.
values = [
    v / values[0] for v in values
] # Normalize the expectation values to evaluate how they decay with distance.

# plotting graph
plt.plot(data, values, marker="o", label="100-qubit GHZ state")
plt.xlabel("Distance between qubits $i$")
plt.ylabel(r"$\langle Z_i Z_0 \rangle / \langle Z_1 Z_0 \rangle$")
plt.legend()
plt.show()
```

✓ 1.7s



In the previous scenario's implementation, we have explored the basic functionality of quantum computing in its most rudimentary form. The guidelines and code presented by the IBM Quantum platform and Qiskit have allowed us to establish a foothold in the steep learning curve inherent to the developing quantum information systems environment. In following the steps to implement a "Hello Quantum World", we have entangled qubits using both simulators and physical quantum hardware.

The complexity of these tasks is masked by the simplicity of following the Qiskit code provided by the IBM Quantum Platform, but even this stepwise exploration revealed a bit of a challenge. Due to the speed with which the field of quantum computing and quantum information systems are developing, I had to chase down and reimplement code that had been deprecated, and frequently found myself adjusting both the recommended Qiskit code, and the depth of my understanding in order to replicate what would be an incredibly simple, introductory coding exercise in the world of classical computing. The difference in execution between a simple "Hello World" in Python and "Hello Quantum World" is staggering,

Contextualizing Quantum Resistant Encryption and Potential Threats

Security Considerations within the Quantum Environment:

The National Quantum Initiation Act of 2018 (NQI Act; P.L. 115-368; codified at 15 U.S.C. §§8801 et seq.), defines quantum information science, which is used within the context of the development of quantum information system devices, as “the use of the laws of quantum physics for the storage, transmission, manipulation, computing, or measurement of information.” (Zhu, 2023). This definitive inclusion is notable in the fact that quantum computational advances and advantages are not limited to the acceleration of computational processing, these quantum advances are currently contributing to the development of computing adjacent technologies that involve the measurement of information. From a fundamental perspective, the involvement of quantum computing technology and methodologies in emerging environmental sensor technologies, (Amerongen, 2021), creates defense and security concerns that are equivalent to the challenges currently being presented by the global network of insecure Internet of Things (IoT) devices.

In the event of a security breach involving quantum information systems sensors, intruders will have access to unparalleled amounts of detailed information within the context of that system. The salient point in this scenario is that with increasing computational ability and environmental sensing technology being leveraged through quantum technology, potential threat actors have the opportunity to access these technologies and leverage the secure accessibility issues that plague IoT devices that rely on classical computational principles and potentially apply those to developing quantum devices. For every increase in computational and data processing ability, there is an axiomatic increase in the risk of security vulnerabilities and their potential to cause damage. Post-quantum encryption algorithms and their associated protocols will need to be included in the post-quantum infrastructural overall to the same degree that they are utilized within traditional encrypted communication channels.

This is notable for the purposes of discussion surrounding the creation of cybersecurity principles and mechanisms of action in a post-quantum computing environment; with a diversity of qubit creation and manipulation options available to potential bad actors, the manifold avenues to quantum computation contribute to an engineering environment where quantum devices would be less limited to budgeting and resource access and acquisition issues. This presents future security issues that mirror the struggle to contain nuclear weapons development in destabilized nations and deterring non-state acting security threat groups from accessing the necessary materials to develop rudimentary nuclear weapons.

While the comparison to nuclear technology stands from a geopolitical security perspective, the development of quantum computing is also an antithesis to nuclear security protocols in the fact that the aforementioned diversity of pathways to qubit development again

allows for a less constrained engineering environment; while threat-actors attempting to develop nuclear weapons are deterred by material access issues and the complex and expensive utilization of a general mechanical developmental pipeline to fission (Laserson, 2018), disruptive quantum computing devices will possibly be developed a myriad of different ways, at different budget scales, in the near future.

A separate potential threat could develop in tandem with the previously mentioned vulnerability scenario and is essentially a risk-laden inverse; a global internet structure that relies on quantum information systems could be especially vulnerable due to its inherent fragility and susceptibility to interference with qubit entanglement. In fact, according to the Massachusetts Institute of Technology review, an entire future system could be thrown into disarray by the injection of random information or “noise” (Emerging Technology from the arXiv, 2024). Further research has suggested that a successful crippling of a quantum information system may be possible with only three colluding individuals (Gomez-Ruiz, 2024). This looming possibility is an added encouragement for the development of post-quantum encryption algorithms and protocols (Amerongen, 2021).

Contextualizing Quantum Resistant Encryption and Potential Threats

Security Considerations within the Quantum Environment: Shor’s Algorithm.

The possession of quantum computing devices and quantum information systems is only one part of a future threat actor’s arsenal. In order to leverage the power of quantum computing against our current encryption infrastructure, the processing ability of quantum computing devices must be directed in a manner which renders current encryption standards vulnerable to exposure. This encryption-cracking direction can be provided by a series of algorithms which leverage the unique capabilities of quantum computers, most notably Shor’s algorithm. Shor’s algorithm is an algorithmic approach which is able to compute discrete logarithms and factor large integers within an efficient timescale.

Current encryption standards rely on the difficulty of factoring large integers (RSA) and computing discrete logarithms (ECC) to produce cryptosystems that would take an unfathomably long amount of time to be broken by classical computers. Shor’s algorithm, when implemented by a quantum computing device or quantum information system, is able to break both of these foundational cryptosystems within polynomial time, as opposed to the exponential time required by classical computers to solve this problem (Shor, 1994).

Shor’s algorithm achieves this by reducing a factoring problem into a period finding problem, which allows a solution to be achieved in polynomial time when computed using the quantum Fourier transform and quantum parallelism. Shor’s algorithm is considered the gold standard for the eventual implementation of quantum computing in breaking our current encryption protocols, as it will break the RSA, ECC, and Diffie-Helman encryption protocols

when implemented by a utility scale quantum computer that has solved the problems of decoherence and error correction with large numbers of qubits (Shor, 1997).

In the following segments, I worked with Claude 3.7 Sonnet to implement a “toy” execution of Shor’s algorithm on Qiskit’s Aer quantum simulator, which factors the number 15 using the general principles of Shor’s algorithm (Anthropic, 2025). Due to the possibility of noise and decoherence present when completing this exercise on one of the IBM QPU’s, I decided to forgo the hardware execution and calculate this on the simulator for ease of use and proof of concept.

Developing this code in tandem with Claude 3.7 Sonnet was more of an exercise in prompt-engineering and patching deprecated code than it was generating novel code. This code predominantly pulls from older iterations of IBM’s Quantum Platform and the Qiskit Textbook, which is referenced in the citations list.

```
import numpy as np
import math
from fractions import Fraction
from qiskit import QuantumCircuit
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
#Zach Bradley Post-Quantum Encryption Algorithms
#Created with the assistance of Claude 3.7 Sonnet.
# Helper functions for Shor's algorithm
def gcd(x, y):
    """Calculate the greatest common divisor of x and y."""
    while y:
        x, y = y, x % y
    return x

def continued_fraction(x, y, max_depth=20):
    """Convert x/y to a continued fraction representation."""
    fractions = []

    for _ in range(max_depth):
        quotient = x // y
        fractions.append(quotient)
        remainder = x % y

        if remainder == 0:
            break

        x, y = y, remainder
```

```

    return fractions

def convergents(fractions):
    """Compute the convergents from a continued fraction representation."""
    convs = []

    numerator_prev2, denominator_prev2 = 1, 0
    numerator_prev1, denominator_prev1 = fractions[0], 1

    convs.append((numerator_prev1, denominator_prev1))

    for i in range(1, len(fractions)):
        numerator = fractions[i] * numerator_prev1 + numerator_prev2
        denominator = fractions[i] * denominator_prev1 + denominator_prev2

        convs.append((numerator, denominator))

        numerator_prev2, denominator_prev2 = numerator_prev1, denominator_prev1
        numerator_prev1, denominator_prev1 = numerator, denominator

    return convs

def find_period(a, N):
    """
    Find the period of  $a^x \bmod N$  using quantum phase estimation.
    This is a simplified version for demonstration purposes.
    """
    # Determine the number of qubits needed
    n_count = math.ceil(math.log2(N**2)) # For phase estimation
    n_input = math.ceil(math.log2(N))    # For modular exponentiation

    # Create quantum circuit
    qc = QuantumCircuit(n_count + n_input, n_count)

    # Initialize input register to  $|1\rangle$ 
    qc.x(n_count)

    # Apply Hadamard gates to counting qubits
    for i in range(n_count):
        qc.h(i)

    # Apply controlled unitary operations
    for i in range(n_count):
        # Apply  $a^{(2^i)} \bmod N$  to the target register
        power = (2**i)

```



```

power_mod = pow(a, power, N)

# This is a simplified implementation
# In reality, we would construct a circuit for modular exponentiation
# For this toy example, we'll add a controlled phase rotation
for j in range(n_input):
    target_bit = (n_count + j)
    # Apply controlled phase based on power_mod
    phase = (2 * np.pi * power_mod * (2**j)) / N
    qc.cp(phase, i, target_bit)

# Apply inverse QFT to the counting register
for i in range(n_count//2):
    qc.swap(i, n_count-i-1)

for i in range(n_count):
    qc.h(i)
    for j in range(i+1, n_count):
        phase = -np.pi / (2**(j-i))
        qc.cp(phase, j, i)

# Measure the counting register
qc.measure(range(n_count), range(n_count))

# Execute the circuit on the simulator
simulator = Aer.get_backend('qasm_simulator')
result = simulator.run(qc, shots=1024).result()
counts = result.get_counts()

# Process the measurement results
measured_phases = []
for outcome, count in counts.items():
    decimal_outcome = int(outcome, 2)
    phase = decimal_outcome / (2**n_count)
    measured_phases.append((phase, count))

# Sort phases by count (most frequent first)
measured_phases.sort(key=lambda x: x[1], reverse=True)

# Attempt to find the period from the most likely phase measurement
for phase, _ in measured_phases:
    # Convert the phase to a fraction
    frac = Fraction(phase).limit_denominator(N)

    # The denominator might be the period or a factor of it

```

```

        r = frac.denominator

        # Check if this is actually the period
        if r > 1 and pow(a, r, N) == 1:
            return r

    return None

def shors_algorithm(N):
    """
    Implement Shor's algorithm to factor N.
    This is a simplified version for demonstration purposes.
    """

    # Check if N is even
    if N % 2 == 0:
        return 2, N // 2

    # Check if N is a prime power
    for i in range(2, int(math.log2(N)) + 1):
        root = round(N**(1/i))
        if root**i == N:
            return root, root**(i-1)

    # Choose a random number a < N
    import random
    a = random.randint(2, N-1)

    # Check if a and N share a common factor
    g = gcd(a, N)
    if g > 1:
        return g, N // g

    # Use the quantum part of Shor's algorithm to find the period
    r = find_period(a, N)

    if r is None:
        return "Failed to find the period."

    # If r is odd, try again
    if r % 2 != 0:
        return "r is odd, try again with a different 'a'."

    # Compute candidates for factors
    x = pow(a, r // 2, N)

```

```

    if x == N - 1:
        return "Bad luck, try again with a different 'a'."

    # Calculate factors
    factor1 = gcd(x + 1, N)
    factor2 = gcd(x - 1, N)

    if factor1 == 1 or factor1 == N:
        return factor2, N // factor2
    else:
        return factor1, N // factor1

# Example usage: factor 15
N = 15
print(f"Attempting to factor N = {N}")

# For demonstration purposes, let's manually set a = 7
a = 7
print(f"Using a = {a}")

# Check if gcd(a, N) > 1
g = gcd(a, N)
if g > 1:
    print(f"Lucky! gcd({a}, {N}) = {g}")
    print(f"Factors of {N}: {g} and {N//g}")
else:
    print(f"gcd({a}, {N}) = 1, proceeding with the algorithm")

# Find the period of a^x mod N
r = find_period(a, N)
print(f"The period r = {r}")

if r is not None and r % 2 == 0:
    x = pow(a, r // 2, N)
    print(f"a^(r/2) mod N = {x}")

    if x != N - 1:
        factor1 = gcd(x + 1, N)
        factor2 = gcd(x - 1, N)
        print(f"gcd({x+1}, {N}) = {factor1}")
        print(f"gcd({x-1}, {N}) = {factor2}")

        if factor1 > 1 and factor1 < N:
            print(f"Factors of {N}: {factor1} and {N//factor1}")
        elif factor2 > 1 and factor2 < N:

```

```

        print(f"Factors of {N}: {factor2} and {N//factor2}")
    else:
        print("Failed to find the factors.")
    else:
        print("Bad luck, try again with a different 'a'.")
    else:
        print("r is odd or None, try again with a different 'a'.")

# Run the general algorithm
print("\nRunning the full Shor's algorithm:")
factors = shors_algorithm(N)
print(f"Factors of {N}: {factors}")
(Anthropic, 2025).

```

The results successfully compute the factors of 15 (5,3) using a simplified version of Shor's algorithm

Executing the code block via VS Code and Qiskit.

```

... Attempting to factor N = 15
Using a = 7
gcd(7, 15) = 1, proceeding with the algorithm
The period r = 4
a^(r/2) mod N = 4
gcd(5, 15) = 5
gcd(3, 15) = 3
Factors of 15: 5 and 3

Running the full Shor's algorithm:
Factors of 15: (5, 3)

```

Shor's algorithm is foundational to the concept of Post-Quantum Cryptography in the fact that it is the ultimate antagonist. Given that utility scale quantum computer and quantum information systems are developed with a large enough number of qubits, a correctly leveraged implementation of Shor's algorithm will completely remove efficacy of our current cryptographic protocols, systems, and implementations. Recently, researchers and developers have opined upon and projected various scenarios that may occur on what is now called "Q-Day", or the day that quantum computers and quantum information systems have sufficiently advanced to the point that they are able to crack modern cryptographic infrastructure protocols.

In an article for Wired magazine by Amit Katwala, the impact of Q-day is summed up succinctly in the statement “on Q-Day, everything could become vulnerable, for everyone: emails, text messages, anonymous posts, location histories, bitcoin wallets, police reports, hospital records, power stations, the entire global financial system” (Katwala, 2025). The impact of quantum computers and quantum information systems, armed with Shor’s algorithm, will be vastly destructive across these specific vulnerabilities, and many others.

Contextualizing Quantum Resistant Encryption and Potential Threats

An Expansion of The Apple Orchard: A Visual Exploration of Quantum Algorithms.

In the interest of continuing our visual exploration of quantum computing and post-quantum cryptographic algorithms through metaphor, we will return to The Apple Orchard, where we last envisioned the farmer leveraging the power of quantum mechanics to multiply himself across the orchard for instantaneous distributed apple picking. We will incorporate the concepts of cryptography into our Apple Orchard metaphor in order to further explore the implications of quantum computing on classical cryptographic systems versus post-quantum cryptographic algorithms.

Consider the Apple Orchard at night. A rouge intruder, who would like to steal some apples, has arrived at the Orchard to abscond with some bounty. Unfortunately for him, there is a significant restriction. The farmer has decided to encircle the apple trees with tall wicker cages that serve to restrict the access of deer and other unwanted diners. Each cage is secured by a key lock to further protect the integrity of the doorway. In this metaphorical structure, the key locks are representative of modern general password encryption. In this scenario, in order for the intruder to break through the fencing, he must employ some sort of lock-picking strategy to crack the lock and access the apples.

It may be worth the time of the intruder to pick the lock off of one apple tree if he was simply hungry himself, but to make a profit off of his illegal haul, he must pick the locks across the entire orchard and harvest the apples. Even a marginally simple lock to pick would require a significant time investment when distributed across a number of apple trees, but the intruder could affectively pick the locks fast enough over-night to make it worth his while. This metaphorical scenario reflects the deterrence properties of general encryption.

Let’s expand our metaphor just a little bit further in order to explore the impact of modern cryptographic protocols. The farmer has been dealing with the issue of thieves again in his Orchard, and the simple key locks that protect the wicker fencing have proven insufficient. The farmer has decided to modernize and has replaced the key lock with four- digit combination locks. When the intruder returns at night with his lock-picking kit, he is presented with a bit of a surprise. Given that the new locks are 4-digit combination locks with digits ranging from 0000 to 9999, the number of combinations available to the owner are 10^4 or 10,000 possible

combinations. At one lock, with a manual combination attempt taking around five seconds, it would take around fourteen hours for the intruder to break the combination on average.

The time that the intruder would spend attempting to brute force a single combination lock would take almost double the time of the average work-day for access to one tree, which is certainly not an affective usage of thieving time. This is an example of a classical computer attempting to brute-force a modern encryption protocol such as RSA, AES, or ECC. It is feasible within the scope of cosmological eons, but not practical for the day-to-day activities of an intruder. It is for this reason that these encryption protocols have been utilized to such an extent, as they have been practically iron-clad.

Now we will factor in the metaphorical impacts of quantum computing into our orchard scenario once again. This time, the intruder also woke up feeling strange, and was able to multiply himself across the orchard using the power of quantum physical phenomena and entanglement at night. While his individual replicants all faced the combination locks present on the wicker fences, they worked tirelessly all night to brute force the combination locks. They needed no breaks and were able to leverage the ability to try many combinations at once, even on individual locks. With their newfound powers they were able to abscond with every single apple in The Apple Orchard by daybreak. This visual exercise allegorically describes the impact of quantum computers, when effectively utilizing a decryption process like Shor's algorithm to decrypt even the most robust cryptographic protocols within the bounds of realistic timeframes.

While the metaphorical exploration of quantum computing in The Apple Orchard allegory is a significant simplification and generalization of the topic, quantum physicists have a long and colorful history of using metaphor and visual storytelling in order to convey the underlying message of such a complex, perplexing, and counterintuitive scientific topic. It seems appropriate to attempt the same strategy in exploring these complex technological and scientific topics within the context of cybersecurity.

Richard Feynman, Neils Bohr, Erwin Schrodinger, and Wolfgang Pauli in particular relished the value of wordplay and symbolic language in their forays into the quantum realm. Even Einstein employed a metaphorical reference in his famous rebuke of probabilistic interpretation in quantum physics, stating in a letter to Max Born, "God does not play dice with the Universe" (Einstein & Born, 1971/1926, p. 90). While this is a paraphrase of the interpretation that translated Einstein's German letter into English, it captures the sentiment that Einstein was conveying regarding the subject.

Contextualizing Quantum Resistant Encryption and Potential Threats

Security Considerations within the Quantum Environment: The Deutsch-Jozsa Algorithm.

While Shor's algorithm is widely considered to be the dominant standard for the implementation of quantum decryption principles (Mosca & Piani 2025), It is not the only algorithm that can be used to decrypt modern encryption protocols when properly implemented on a utility grade quantum computer. Preceding Shor's algorithm in the timeline of quantum cryptographic development is a theoretical proto-quantum algorithm known as the Deutsch-Jozsa algorithm (Deutsch & Jozsa, 1992). This algorithm was brought to my attention by Dr. Murthy Rallapalli, who has written a paper and accompanying code specifically for the implementation of the Deutsch-Jozsa algorithm on the IBM Quantum platform using QASM (Quantum Assembly Language).

Operating predominantly as a theoretical and eventually pedagogical exercise within the context of quantum cryptography, the Deutsch-Jozsa algorithm “was the first example of a quantum algorithm that performs better than the best classical algorithm. It showed that there can be advantages to using a quantum computer as a computational tool for a specific problem” (Qiskit, 2023). The Deutsch-Jozsa algorithm originated to supply a simple demonstrable example of provable quantum speedup over classical computational algorithms in certain settings.

To explore the operational principles of the Deutch-Jozsa algorithm, a hidden Boolean function is postulated which returns either 0 or 1 for an input that is a string of bits. The function executes in a manner that is guaranteed to be either constant or balanced. At its most rudimentary form it can be expressed in the pseudo functional manner (Qiskit, 2023).

$$f(\{x_0, x_1, x_2, \dots\}) \rightarrow 0 \text{ or } 1 \text{ where } x_n \text{ is } 0 \text{ or } 1$$

The function is constant if it returns either all 1-bit values or all 0-bit values regardless of input. A function is balanced if it returns 1's for exactly half of the inputs and 0's for the remaining half, or vice versa. The challenge with this example is to determine whether the function is balanced or constant utilizing computational methods.

Given the opportunity to process this using classical computational methods, an answer confirming a balanced function can theoretically be revealed in two input queries. If one input reveals a 0 as an output and the next input reveals a 1 as an output, then we can determine that the function is balanced. If the function reveals a continuous resultant stream of 0's or 1's as output, then we will have to test, in the worst-case scenario, up to $2^{n-1} + 1$ inputs to confirm that the function is constant (Qiskit, 2023).

If we continue with our process using quantum computational methods and an appropriately structured Deutsch-Jozsa circuit, the functions state, operating as either constant or balanced, can be determined in exactly one call. When passed to a quantum oracle, the inputs are

evaluated by leveraging qubit entanglement and superposition to provide simultaneous computation.

In the next steps, we will execute a variation of the Deutsch-Jozsa algorithm via the Qiskit Aer quantum simulator. This implementation will test an input of three bits.

```
from qiskit import QuantumCircuit
from qiskit_aer import Aer # For simulator backend
from qiskit.visualization import plot_histogram

#Zach Bradley implementation of Deutsch-Jozsa using Deepseek and IBM's Qiskit textbook.

# Define oracles
def constant_oracle(qc):
    qc.x(3)

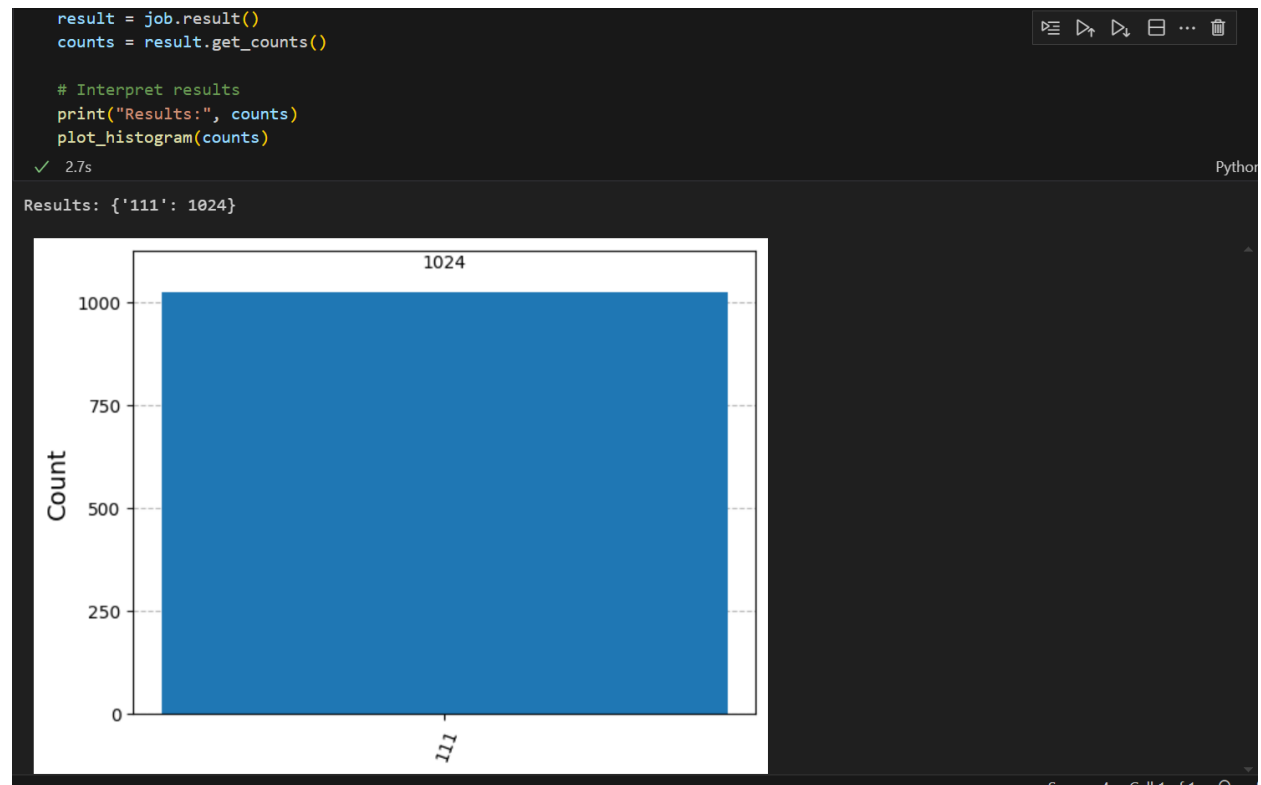
def balanced_oracle(qc):
    qc.cx(0, 3)
    qc.cx(1, 3)
    qc.cx(2, 3)

# Build circuit
def deutsch_jozsa(oracle_func, n=3):
    qc = QuantumCircuit(n+1, n)
    qc.x(n)
    qc.h(range(n+1))
    oracle_func(qc)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc

# Run simulation
dj_circuit = deutsch_jozsa(balanced_oracle)
simulator = Aer.get_backend('qasm_simulator')

# New style execution
job = simulator.run(dj_circuit, shots=1024) # Replaces execute()
result = job.result()
```


The results and a histogram plot of the results reveal that our black-box function is constant. We received a single state of '111' across all 1024 shots which indicates that the tested oracle was functioning as a constant 1 black-box. This experimental implementation demonstrates quantum superiority in the fact that the Deutsch-Jozsa algorithm required only one quantum query, which is exponentially faster than the worst-case test-scenario of $2^{n-1} + 1$ evaluations for n-bit inputs to confirm that the function is constant using a classical algorithm.



Let's also execute this same function using actual IBM QPU's. I have worked with Claude 3.7 Sonnet to request assistance in executing the Deutsch-Josza Algorithm on IBM's QPU's (Anthropic, 2025).

```
... Qiskit version: 1.4.0
Qiskit IBM Runtime version: 0.36.1

# Import necessary libraries
#Zach Bradley implementing the Deutsch-Josza Algorithm with assistance from Claude
from qiskit import QuantumCircuit, transpile
from qiskit_ibm_runtime import QiskitRuntimeService, Session, Sampler
import numpy as np

# Load your IBM Quantum account - this assumes your API key is already saved
service = QiskitRuntimeService()

# List available backends
available_backends = service.backends()
real_backends = [backend for backend in available_backends if not backend.simulator]
print("Available systems:", [backend.name for backend in real_backends])

if not real_backends:
    print("No real quantum systems available. Please check your account access.")
    # Fallback to a simulator for testing
    backend_name = 'ibmq_qasm_simulator'
    backend = service.backend(backend_name)
else:
    # Choose the first available real backend
    backend = real_backends[0]
    print(f"Selected backend: {backend.name}")

# Define functions for the oracles
def constant_function_oracle(qc, qubits):
    # For constant function returning 0 for all inputs - do nothing
```

[31]

```
# Create a Deutsch-Jozsa circuit with a suitable number of qubits for the hardware
n = 2

# Test with both constant and balanced functions
constant_circuit = deutsch_jozsa_circuit(n, constant_function_oracle)
balanced_circuit = deutsch_jozsa_circuit(n, balanced_function_oracle)

# Print original circuits
print("Original constant function circuit:")
print(constant_circuit.draw())
print("\nOriginal balanced function circuit:")
print(balanced_circuit.draw())

# CRITICAL STEP: Transpile the circuits for the target backend
# This ensures the circuits use gates and connectivity supported by the hardware
print(f"\nTranspiling circuits for {backend.name}...")
constant_circuit_transpiled = transpile(constant_circuit, backend, optimization_level=3)
balanced_circuit_transpiled = transpile(balanced_circuit, backend, optimization_level=3)

# Print transpiled circuits
print("Transpiled constant function circuit:")
print(constant_circuit_transpiled.draw())
print("\nTranspiled balanced function circuit:")
print(balanced_circuit_transpiled.draw())

print(f"Preparing to submit jobs to {backend.name}...")

# Try opening a session with the backend object
with Session(backend=backend) as session:
    # Within the session, create a Sampler (no parameters)
    sampler = Sampler()
```

```
# Submit jobs using the sampler with the TRANSPILED circuits
print("Submitting jobs...")
constant_job = sampler.run([constant_circuit_transpiled], shots=1024)
balanced_job = sampler.run([balanced_circuit_transpiled], shots=1024)

print(f"Constant function job ID: {constant_job.job_id}")
print(f"Balanced function job ID: {balanced_job.job_id}")

print("Jobs submitted to queue. They will run on the quantum hardware.")
print("You can check their status with:")
print("constant_job.status()")
print("balanced_job.status()")

# Uncomment this section if you want to wait for the jobs to complete
# But note that this might take a while due to queue times
"""
print("\nWaiting for jobs to complete (this may take a while)...")
constant_result = constant_job.result()
balanced_result = balanced_job.result()

# Process and display results
print("Jobs completed!")

# Process results based on the expected format
try:
    # Try accessing quasi_dists (newer versions)
    constant_quasi_dist = constant_result.quasi_dists[0]
    balanced_quasi_dist = balanced_result.quasi_dists[0]

    # Convert to readable format
    constant_counts = {format(int(k), f'0{n}b'): v for k, v in constant_quasi_dist.items()}
    balanced_counts = {format(int(k), f'0{n}b'): v for k, v in balanced_quasi_dist.items()}
```

```

except (AttributeError, KeyError):
    # Alternative: access counts directly
    constant_counts = constant_result.get_counts(0)
    balanced_counts = balanced_result.get_counts(0)

print("Constant function results:", constant_counts)
print("Balanced function results:", balanced_counts)

# Analyze results
# For a constant function, we expect to measure all zeros with high probability
is_constant_zeros = '00' in constant_counts and constant_counts['00'] > 0.8 * 1024

# For a balanced function, we expect non-zero measurements with high probability
is_balanced_non_zeros = not ('00' in balanced_counts and balanced_counts['00'] > 0.8 * 1024)

print(f"Constant function appears constant (all zeros): {is_constant_zeros}")
print(f"Balanced function appears balanced (non-zeros): {is_balanced_non_zeros}")
"""

# Provide instructions for after the jobs complete
print("\nWhen the jobs complete, you can get and analyze the results with:")
print("""
# First, retrieve the job using the job ID
from qiskit_ibm_runtime import QiskitRuntimeService
service = QiskitRuntimeService()

# Get the jobs
constant_job = service.job(job_id="YOUR_CONSTANT_JOB_ID") # Replace with the actual job ID
balanced_job = service.job(job_id="YOUR_BALANCED_JOB_ID") # Replace with the actual job ID

# Get results
constant_result = constant_job.result()
balanced_result = balanced_job.result()
""")

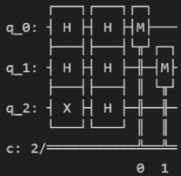
```

Visualizing the original constant function circuits.

```

Available systems: ['ibm_brisbane', 'ibm_sherbrooke']
Selected backend: ibm_brisbane
Original constant function circuit:

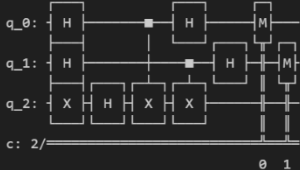
```



```

Original balanced function circuit:

```



Checking the function job status.

```
# Import the service
from qiskit_ibm_runtime import QiskitRuntimeService
service = QiskitRuntimeService()

# Get the job objects using their IDs
constant_job_id = "d03bpjjqnmvg0082apw0" # The constant function job ID from your output
balanced_job_id = "d03bpk2rxz8g008fzf5g" # The balanced function job ID from your output

# Retrieve the jobs
constant_job = service.job(constant_job_id)
balanced_job = service.job(balanced_job_id)

# Check the status
print(f"Constant function job status: {constant_job.status()}")
print(f"Balanced function job status: {balanced_job.status()}")
```

✓ 8.6s Python

Constant function job status: DONE
Balanced function job status: DONE

```
# Get results
constant_result = constant_job.result()
balanced_result = balanced_job.result()

# Print the structure of the results to understand what's available
print("Constant result structure:")
print(dir(constant_result))
print("\nConstant result metadata:")
print(constant_result)

# Try alternative methods to access the results
try:
    # Method 1: Access directly through the result
    if hasattr(constant_result, 'get_counts'):
        constant_counts = constant_result.get_counts()
        balanced_counts = balanced_result.get_counts()
    # Method 2: Access through data attribute
    elif hasattr(constant_result, 'data'):
        constant_counts = constant_result.data.get('counts', {})
        balanced_counts = balanced_result.data.get('counts', {})
    # Method 3: Check if results in different format
    elif hasattr(constant_result, 'results'):
        constant_counts = constant_result.results[0].data.counts
        balanced_counts = balanced_result.results[0].data.counts
    # Method 4: Dictionary-like access
    else:
        print("Trying dictionary access...")
        try:
            constant_counts = constant_result['counts']
            balanced_counts = balanced_result['counts']
        except (KeyError, TypeError):
            print("Dictionary access failed. Showing raw result.")
```

```

>
print("\nConstant function results:", constant_counts)
print("Balanced function results:", balanced_counts)

# If we've successfully extracted counts, analyze them
if isinstance(constant_counts, dict) and isinstance(balanced_counts, dict):
    # For constant function, we expect to measure all zeros with high probability
    is_constant_zeros = '00' in constant_counts and constant_counts['00'] / sum(constant_counts.values()) > 0.8

    # For balanced function, we expect non-zero measurements with high probability
    is_balanced_non_zeros = not ('00' in balanced_counts and balanced_counts['00'] / sum(balanced_counts.values()) > 0.8)

    print(f"Constant function appears constant (all zeros): {is_constant_zeros}")
    print(f"Balanced function appears balanced (non-zeros): {is_balanced_non_zeros}")
except Exception as e:
    print(f"Error accessing results: {e}")
    print("Let's show the raw results for manual inspection:")
    print("\nConstant result:", constant_result)
    print("\nBalanced result:", balanced_result)

[34] ✓ 14s Python

... Constant result structure:
['_annotations_', '_class_', '_class_getitem_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_firstlineno_', '_format_', '_ge

Constant result metadata:
PrimitiveResult([SamplerPubResult(data=DataBin(c=BitArray(<shape=(), num_shots=1024, num_bits=2>)), metadata={'circuit_metadata': {}})], metadata={'exe
Trying dictionary access...
Dictionary access failed. Showing raw result.

Constant function results: PrimitiveResult([SamplerPubResult(data=DataBin(c=BitArray(<shape=(), num_shots=1024, num_bits=2>)), metadata={'circuit_metad
Balanced function results: PrimitiveResult([SamplerPubResult(data=DataBin(c=BitArray(<shape=(), num_shots=1024, num_bits=2>)), metadata={'circuit_metad

```

The results with the balanced function job.

```

▷
# Get the job results
job = service.job('d03bpk2rxz8g008fzf5g') # Your balanced function job
job_result = job.result()

# Print the available data structure to inspect it
print("Job result type:", type(job_result))
print("Job result attributes:", dir(job_result))

# For newer Qiskit versions, you might need to access the first result element
if isinstance(job_result, list) or hasattr(job_result, '__getitem__'):
    first_result = job_result[0]
    print("First result type:", type(first_result))
    print("First result attributes:", dir(first_result))

# Try to access the classical register data
# You may need to replace 'c' with your actual classical register name
if hasattr(first_result, 'data'):
    if hasattr(first_result.data, 'c'):
        counts = first_result.data.c.get_counts()
        print("Measurement counts:", counts)
    else:
        print("Available data attributes:", dir(first_result.data))

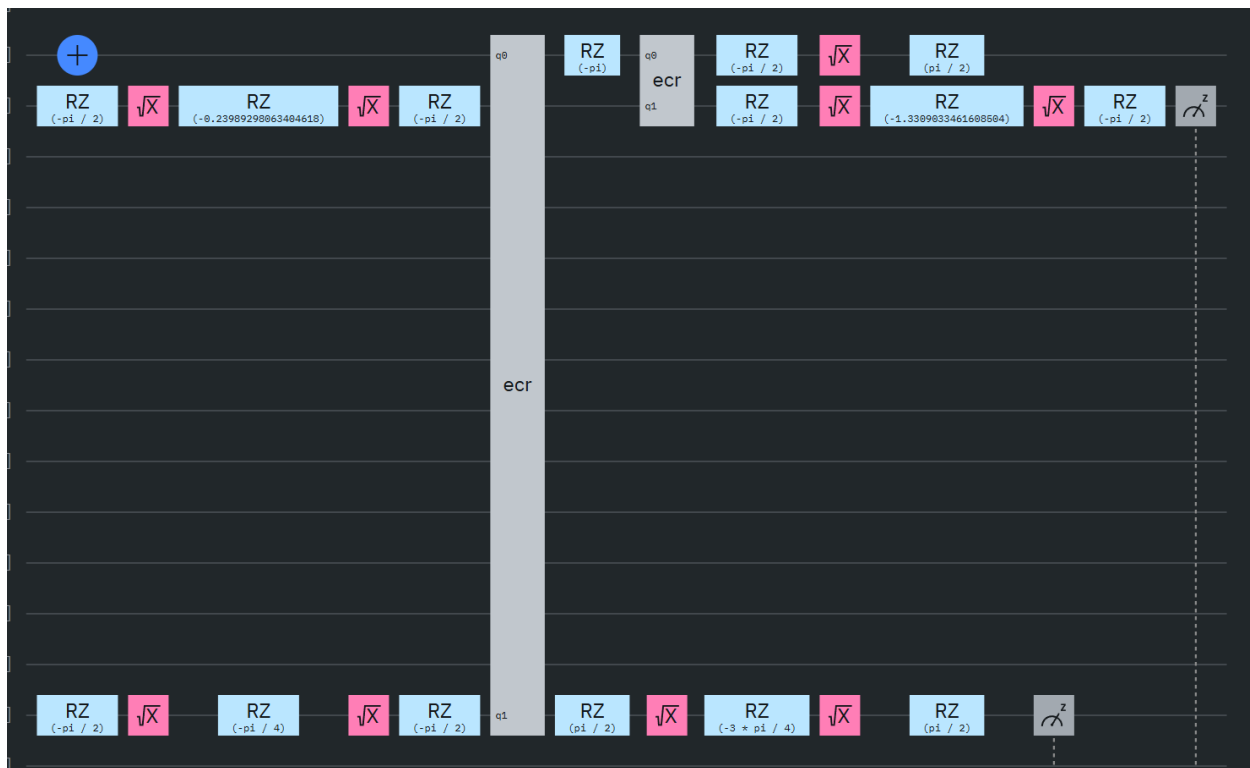
[42] ✓ 22s Python

... Job result type: <class 'qiskit.primitives.containers.primitive_result.PrimitiveResult'>
Job result attributes: ['_annotations_', '_class_', '_class_getitem_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_firstlineno_', '_format_', '_ge
First result type: <class 'qiskit.primitives.containers.sampler_pub_result.SamplerPubResult'>
First result attributes: ['_class_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_firstlineno_', '_format_', '_ge_', '_getattr_
Measurement counts: {'00': 10, '11': 967, '01': 25, '10': 22}

```

The results of the constant function job.

The IBM Quantum circuit diagram.



The constant function test shows the state $|00\rangle$ with 99.5% probability (1019 out of 1024 measurements). This is what is expected for a constant function, where the algorithm should produce the $|00\rangle$ state with high probability. Since we are operating on actual IBM hardware, the results are not 100% accurate due to noise and decoherence.

The balanced function test reveals the state $|11\rangle$ with 94.4% probability, while the $|00\rangle$ state occurs with only 1% probability. This is a balanced function, as the algorithm is designed to produce any state except $|00\rangle$ for balanced functions. Again 100% accuracy isn't possible when interacting with actual quantum hardware versus a simulation.

This implementation was executed and analyzed with assistance from Claude 3.7 Sonnet (Anthropic, 2025). This implementation utilized the IBM Quantum platform and the Qiskit SDK.

Now that we have explored some of the rudimentary functionality that is possible with the Deutsch-Jozsa algorithm and Shor's algorithm, we'll move forward in determining how to effectively eliminate the threat that quantum computing and quantum computing information systems pose to our modern cryptographic infrastructure.

Contextualizing Quantum Resistant Encryption and Potential Defenses

Security Considerations within the Quantum Environment: The Apple Orchard Metaphor

We have explored the implications of quantum computing and quantum information systems within the scope of IBM's Quantum platform by executing a "Quantum Hello World" demonstration, a basic implementation of Shor's Algorithm, and an execution of the Deutsch-Jozsa Algorithm. Throughout our sojourn through the burgeoning quantum computing environment, we have employed The Apple Orchard metaphor to elucidate some of the more complex concepts that form the foundation of quantum computing principles. The question that remains for our research now is, within the metaphorical expression of The Apple Orchard, how can the Farmer protect his apple trees from a thief who possess an ability of multiplicity, much like a quantum computer? How can he safeguard his future apple crops from such advanced thieving practices?

Outside the bounds of our quantum parable, quantum computing is poised to disrupt our entire encryption infrastructure by leveraging the implementation of Shor's algorithm on sufficient utility-grade quantum computing technology. This forthcoming threat will exploit the newfound vulnerability of our most robust encryption protocols (RSA, ECC) and expose countless amounts of data in industries as disparate as finance, defense, social media, telecommunications... any industrial infrastructure that relies on encryption for secure data transfer is vulnerable to the threats poised on Q-day.

In order to accelerate ahead of the curve and prevent disaster, scientists, business leaders, government officials, and IT professionals need to participate in further safeguarding against these novel threats by implementing the post-quantum encryption protocols suggested and provided by the National Institute of Standards and Technology. After a call to action across the industry, and evaluating a number of different suggested algorithms and protocols over a series of rounds, NIST has released its first wave of post-quantum encryption protocols, FIPS 203, 204, and 205.

Contextualizing Quantum Resistant Encryption and Potential Defenses

Security Considerations within the Quantum Environment: FIPS 203

The first of these post-quantum encryption protocols, FIPS 203 is based on the 2017 CRYSTALS-Kyber submission to NIST's call to action (Avanzi et al., 2021). NIST released its final report on the first three finalized post-quantum encryption algorithms (National Institutes of Standards and Technology, 2024), and CRYSTALS-Kyber became FIPS 203: Model-Lattice-Based-Key-Encapsulation Mechanism Standard (National Institutes of Standards and Technology, 2024).

The FIPS 203 Module-Lattice-Based-Key-Encapsulation Mechanism Standard is a key encapsulation mechanism which relies on the computational difficulty of the Module Learning with Errors problem to resist the inquiry of a quantum computer or quantum information system. For reference and review, the FIPS 203 documentation defines a key-encapsulation mechanism (KEM) as a “set of algorithms that, under certain conditions, can be used by two parties to establish a shared secret key over a public channel. A shared secret key that is securely established using a KEM can then be used with symmetric-key cryptographic algorithms to perform basic tasks in secure communications, such as encryption and authentication” (National Institute of Standards and Technology, 2024, p.3).

This initial construction of a public-key channel for the transfer of a private symmetric key is recommended as the quantum-safe replacement for the current Elliptical Curve Diffie-Helman encryption suite. As such, there are three parameter sets of FIPS 203 that are ordered by increasing strength of security complexity and decreased efficiency, ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

The methodology behind ML-KEM requires the generation of a public encapsulation key and a private decapsulation key; both of which facilitate the transfer of a randomly generated secret key which is used to encrypt a cipher text. Utilizing the traditional cryptographic character descriptors Bob, Eve, and Alice, we can describe the ML-KEM protocol as an encryption suite which allows Alice to provide a public encapsulation key, with which Bob uses to encrypt a ciphertext with said randomly generated secret key. Only Alice is able to decrypt the contents of the encrypted “capsule” with her decapsulation key. This is the manner in which Bob and Alice are able to exchange a shared secret symmetric key over a public connection. Later, this shared secret key can be used for quick and efficient encrypted communication between the two parties, using methodologies such as the Advanced Encryption Standard (AES), affectively thwarting any intercepting efforts attempted by the potential “Man in the Middle” attacker, Eve.

This is the traditional public-private key encapsulation methodology that forms the foundation of key exchanges traditionally used in RSA and ECCDH, the difference between these legacy systems and FIPS 203 lies in the generation of the keys using the algorithm that leverages the mathematical complexity of the Module-Learning with Errors problem while generating the encapsulation and de-encapsulation keys. In this scenario, if Eve was capable of intercepting the public key or the ciphertext, she would not be able to derive the private key from those materials without solving the Module Learning with Errors problem, which is affectively unsolvable for both quantum and classical computers alike.

The Module Learning with Errors Problem is the mathematical foundation of the ML-KEM algorithm, and is the facilitating operation that allows FIPS 203 to lead the transition to a post-quantum cryptographic landscape across all digital communication channels. By focusing on the injection of mathematical noise into encryption algorithms via noisy linear algebra over finite fields (Regev, 2005) the Module Learning with Errors Problem introduces noise into the

encryption process which is theoretically intractable without the usage of a secret-key which details the origination of the noise, and the specific deviation applied to the plaintext values. The addition of mathematical noise contributes to both classical and quantum resistance by leveraging the principles of computational hardness present in solving noisy systems, unlike weaker protocols that rely on factoring, such as RSA, or discrete logarithms, such as ECC.

The cornerstone of ML-KEM's mathematical approach is two-fold. The Module Learning with Errors portion is the implementation of linear algebra where error terms are injected into structured equations over finite fields distributed over module lattices, which function as algebraic constructs formed by polynomial rings in n-dimensional space. This serves to merge computational efficiency with geometric difficulty. The injection of error noise serves to boost cryptographic hardness factors, while the module structure allows for efficient operations over the course of twenty-one separate algorithms (National Institute of Standard and Technology, 2024).

$$\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q}$$

where:

- $\mathbf{A} \in R_q^{k \times k}$: Public random matrix (ring elements).
- $\mathbf{s}, \mathbf{e} \in R_q^k$: Secret and error vectors (small coefficients).
- $R_q = \mathbb{Z}_q[X]/(X^n + 1)$: Polynomial ring (e.g., $n = 256, q = 7681$).

Image generated by Deepseek LLM for conciseness of expression in equations (Deepseek, 2024), (National Institute of Standards and Technology, 2024). LaTeX on Microsoft Word was providing difficulties in implementing the correct expression format.

Let's explore some expressions of FIPS 203 via Kali Linux and the Liboqs open-source C library from Open Quantum Safe. We can use Liboqs to perform some benchmarking of the algorithm with full FIPS 203 key exchanges.

To initiate our exploration of FIPS 203 via Liboqs, we'll need to update my base Kali system, install some required libraries and tools, and create a directory structure with a python virtual environment in order to maintain a sense of coherence throughout the project. Additionally, it is important to import and install Liboqs and its associated libraries in the correct locations, so as to avoid dependency issues across the scope of the project.

Some bash commands for initial installation requirements and refreshments include:

sudo apt install build-essential git cmake libssl-dev python3-pip

Commands for installing Liboqs.

```
cd ~/pqc-research/tools
```

```
git clone https://github.com/open-quantum-safe/liboqs.git
```

```
cd liboqs
```

```
mkdir build
```

```
cd build
```

```
cmake ..
```

```
make
```

```
make test
```

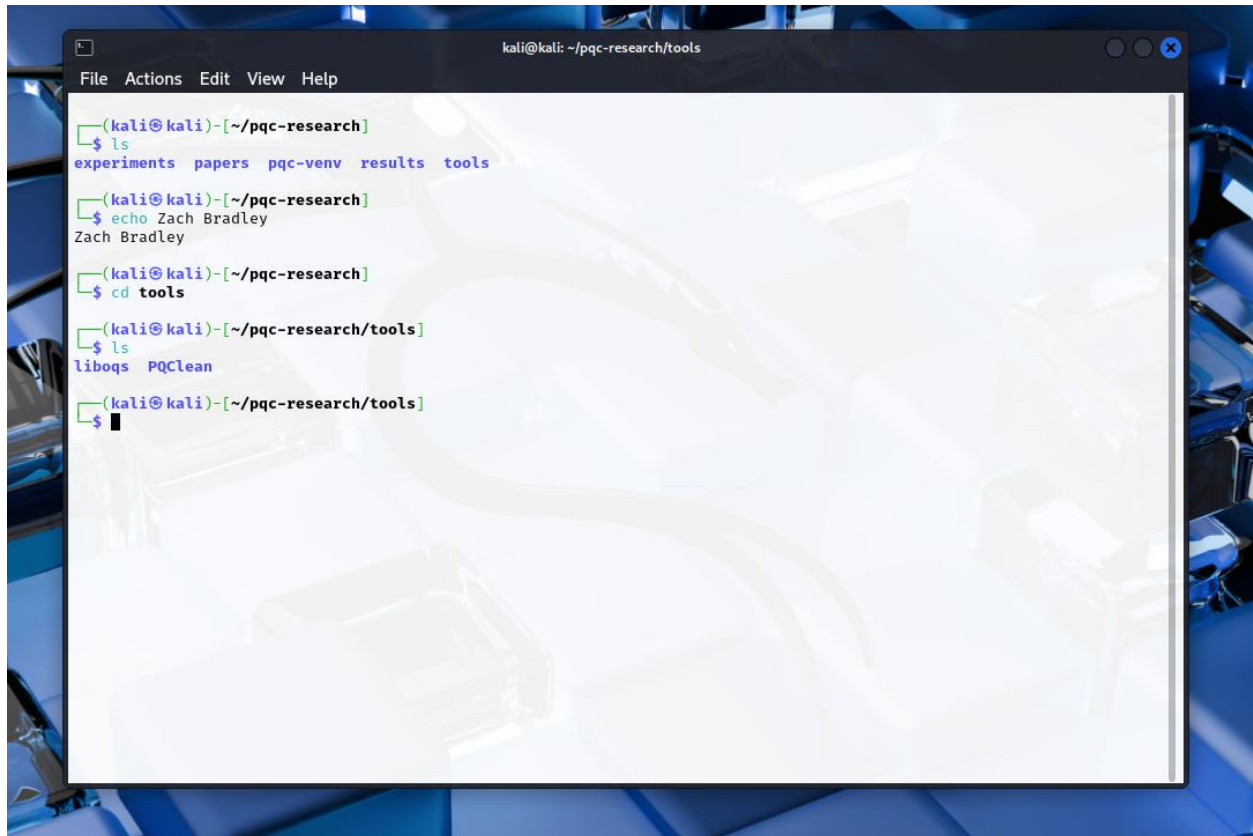
We will also install PQClean within our tools folder in order to maintain clean implementations of the Post-Quantum Cryptography protocols.

```
cd ~/pqc-research/tools
```

```
git clone https://github.com/PQClean/PQClean.git
```

```
cd PQClean
```

An image of the associated directory structure in my Kali machine.

A terminal window titled 'kali@kali: ~/pqc-research/tools' with a menu bar (File, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
(kali@kali)-[~/pqc-research]
$ ls
experiments  papers  pqc-venv  results  tools

(kali@kali)-[~/pqc-research]
$ echo Zach Bradley
Zach Bradley

(kali@kali)-[~/pqc-research]
$ cd tools

(kali@kali)-[~/pqc-research/tools]
$ ls
liboqs  PQClean

(kali@kali)-[~/pqc-research/tools]
$
```

Implementing a FIPS 203 CRYSTALS-Kyber test using Liboqs.

Generating C code for the Kyber exchange in Nano utilizing Claude 3.7 Sonnet for general structure and speed of development.

```
kali@kali: ~/pqc-research/experiments/pqc_testing/key_exchange
File Actions Edit View Help
GNU nano 8.3 kyber_test.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <oqs/oqs.h>

int main(void) {
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *ciphertext = NULL;
    uint8_t *shared_secret_alice = NULL;
    uint8_t *shared_secret_bob = NULL;
    OQS_STATUS ret = OQS_ERROR;

    // Initialize the Kyber algorithm
    OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_kyber_768);
    if (kem == NULL) {
        printf("Error: couldn't initialize Kyber-768\n");
        return EXIT_FAILURE;
    }

    printf("Successfully initialized Kyber-768\n");
    printf("Public key length: %zu bytes\n", kem->length_public_key);
    printf("Secret key length: %zu bytes\n", kem->length_secret_key);
    printf("Ciphertext length: %zu bytes\n", kem->length_ciphertext);
    printf("Shared secret length: %zu bytes\n", kem->length_shared_secret);

    // Allocate memory
    public_key = malloc(kem->length_public_key);
    secret_key = malloc(kem->length_secret_key);
    ciphertext = malloc(kem->length_ciphertext);
    shared_secret_alice = malloc(kem->length_shared_secret);
    shared_secret_bob = malloc(kem->length_shared_secret);

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

(Anthropic, 2025) (Open Quantum Safe, 2025)

```
kali@kali: ~/pqc-research/experiments/pqc_testing/key_exchange
File Actions Edit View Help
GNU nano 8.3 kyber_test.c
shared_secret_bob = malloc(kem->length_shared_secret);

if ((public_key == NULL) || (secret_key == NULL) || (ciphertext == NULL) ||
    (shared_secret_alice == NULL) || (shared_secret_bob == NULL)) {
    printf("Error: malloc failed\n");
    ret = OQS_ERROR;
    goto err;
}

// Generate keypair
ret = OQS_KEM_keypair(kem, public_key, secret_key);
if (ret != OQS_SUCCESS) {
    printf("Error: keypair generation failed\n");
    goto err;
}

// Encapsulation
ret = OQS_KEM_encaps(kem, ciphertext, shared_secret_bob, public_key);
if (ret != OQS_SUCCESS) {
    printf("Error: encapsulation failed\n");
    goto err;
}

// Decapsulation
ret = OQS_KEM_decaps(kem, shared_secret_alice, ciphertext, secret_key);
if (ret != OQS_SUCCESS) {
    printf("Error: decapsulation failed\n");
    goto err;
}

// Compare shared secrets
if (memcmp(shared_secret_alice, shared_secret_bob, kem->length_shared_secret) != 0) {
    printf("Error: shared secrets do not match\n");
    ret = OQS_ERROR;
    goto err;
}

err:
// Clean up
if (public_key) free(public_key);
if (secret_key) free(secret_key);
if (ciphertext) free(ciphertext);
if (shared_secret_alice) free(shared_secret_alice);
if (shared_secret_bob) free(shared_secret_bob);
OQS_KEM_free(kem);

return (ret == OQS_SUCCESS) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

```
kali@kali: ~/pqc-research/experiments/pqc_testing/key_exchange
File Actions Edit View Help
GNU nano 8.3 kyber_test.c
    goto err;
}

// Decapsulation
ret = OQS_KEM_decaps(kem, shared_secret_alice, ciphertext, secret_key);
if (ret != OQS_SUCCESS) {
    printf("Error: decapsulation failed\n");
    goto err;
}

// Compare shared secrets
if (memcmp(shared_secret_alice, shared_secret_bob, kem->length_shared_secret) != 0) {
    printf("Error: shared secrets do not match\n");
    ret = OQS_ERROR;
    goto err;
}

printf("Success! Alice and Bob generated the same shared secret.\n");
ret = OQS_SUCCESS;

err:
// Clean up
if (public_key) free(public_key);
if (secret_key) free(secret_key);
if (ciphertext) free(ciphertext);
if (shared_secret_alice) free(shared_secret_alice);
if (shared_secret_bob) free(shared_secret_bob);
OQS_KEM_free(kem);

return (ret == OQS_SUCCESS) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Code developed between myself and Claude 3.7 Sonnet over the course of a dialogue regarding proper testing of FIPS 203 using the Liboqs library (Anthropic, 2025).

Now we will need to change our focus to creating a compilation script for the execution of our FIPS 203 (CRYSTALS-Kyber) implementation.

Creating a compile.sh script.



```
File Actions Edit View Help
GNU nano 8.3 compile.sh
#!/bin/bash

# Detect the liboqs directory
LIBOQS_DIR=~/.pqc-research/tools/liboqs

# Compile Kyber test
gcc -o pqc_testing/key_exchange/kyber_test pqc_testing/key_exchange/kyber_test.c \
    -I${LIBOQS_DIR}/build/include -L${LIBOQS_DIR}/build/lib -loqs -lcrypto -ldl

echo "Compilation complete. Set the library path before running:"
echo "export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:${LIBOQS_DIR}/build/lib"
```

[Read 11 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

(Anthropic, 2025) with guidance from myself and implementations suggested by Open Quantum Safe (Open Quantum Safe, 2025).


```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
└─$ cd key_exchange
└─(kali@kali)-[~/pqc-research/experiments/pqc_testing/key_exchange]
└─$ nano kyber_test.c
└─(kali@kali)-[~/pqc-research/experiments/pqc_testing/key_exchange]
└─$ ..
└─(kali@kali)-[~/pqc-research/experiments/pqc_testing]
└─$ ..
└─(kali@kali)-[~/pqc-research/experiments]
└─$ ..
└─(kali@kali)-[~/pqc-research]
└─$ cd experiments
└─(kali@kali)-[~/pqc-research/experiments]
└─$ nano compile.sh
└─(kali@kali)-[~/pqc-research/experiments]
└─$ chmod +x compile.sh
└─(kali@kali)-[~/pqc-research/experiments]
└─$ echo Zach Bradley
Zach Bradley
└─(kali@kali)-[~/pqc-research/experiments]
└─$ cd ~/pqc-research/experiments
└─./compile.sh
Compilation complete. Set the library path before running:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/kali/pqc-research/tools/liboqs/build/lib
└─(kali@kali)-[~/pqc-research/experiments]
└─$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/pqc-research/tools/liboqs/build/lib
└─./pqc_testing/key_exchange/kyber_test
```

We've gone ahead and created the compilation script and set the library path so the dependencies are set.

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help

(kali@kali)-[~/pqc-research]
$ cd experiments

(kali@kali)-[~/pqc-research/experiments]
$ nano compile.sh

(kali@kali)-[~/pqc-research/experiments]
$ chmod +x compile.sh

(kali@kali)-[~/pqc-research/experiments]
$ echo Zach Bradley
Zach Bradley

(kali@kali)-[~/pqc-research/experiments]
$ cd ~/pqc-research/experiments
./compile.sh
Compilation complete. Set the library path before running:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/kali/pqc-research/tools/liboqs/build/lib

(kali@kali)-[~/pqc-research/experiments]
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/pqc-research/tools/liboqs/build/lib
./pqc_testing/key_exchange/kyber_test
Successfully initialized Kyber-768
Public key length: 1184 bytes
Secret key length: 2400 bytes
Ciphertext length: 1088 bytes
Shared secret length: 32 bytes
Success! Alice and Bob generated the same shared secret.

(kali@kali)-[~/pqc-research/experiments]
$ echo Zach Bradley
Zach Bradley

(kali@kali)-[~/pqc-research/experiments]
$ █
```

In this successful key exchange, we executed a C program which utilized the Liboqs library to implement an execution of ML-KEM 768.

In this scenario the public key-length generated by our “Alice” is 1184 bytes long.

The secret key-length (private key): 2400 bytes

Length of the ciphertext generated by “Bob”: 1088 bytes.

Shared secret key: 32 bytes.

This successful key-generation will serve to dissuade an attack by a classical or quantum Eve.

In comparison to alternative classical cryptographic methods, such as RSA, the key lengths are fairly large, at times approaching 2-5 times larger key lengths (e.g., 1,184-byte public keys for Kyber-768 versus 384-byte keys for RSA-3072). Apart from the processing of the module lattice polynomial matrices in the public and private keys, the size of the ciphertext is expanded by the addition of intentional error noise (National Institute of Standards and Technology, 2024). These increased key-lengths are a trade-off that is inescapable when considering the utility of ML-KEM as a cryptographic protocol that is resistant to both classical, and eventually quantum computers.

Contextualizing Quantum Resistant Encryption and Potential Defenses

Security Considerations within the Quantum Environment: FIPS 204

The National Institute of Standards and Technology has suggested two additional post-quantum cryptographic protocols to accompany FIPS 203 ML-KEM during the Department of Commerce’s initial official release for post-quantum encryption standards. FIPS 204 ML-DSA is Module Lattice – Digital Signature Algorithm, the second NIST algorithm released for the post-quantum cryptography standardization project which includes an algorithmic structure specifically designed for the generation of digital signatures using Module Learning with Errors (MLWE) and the Module Short Integer Solution (MSIS) when expressed over the context of module lattice problems (NIST, 2024). This execution in particular involves Fiat-Shamir heuristics within its expression of MLWE and MSIS; the ML-DSA is “is a Schnorr-like signature with several optimizations” (NIST p. 19, 2024).

Unlike the FIPS 203 ML-KEM (CRYSTALS-Kyber) Key Encapsulation Mechanism which relies exclusively on the Module Learning with Errors over module lattices, the foundational operational principles of ML-DSA utilizes both MLWE and MSIS in its implementation. FIPS 203 was constructed and will be implemented predominantly for secret key exchange protocols, while FIPS 204 will be utilized for the reinforcement of digital certificates. With the addition of the problem of Module Short Integer Solutions to Module Learning with Errors, FIPS 204 ML-DSA is able to manifest the functional impossibility of solving a short-vectors path problem without an initial known key structure. This, in theory will provide such immense difficulty that even a sufficiently powerful and functional quantum computer will be unable to decrypt and forge a matching digital signature.

Simplified MSIS Equation

Given a public matrix A over the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, find a **non-zero short vector** $\mathbf{z}$ such that:

$$A \cdot \mathbf{z} = \mathbf{0} \pmod{q},$$

Constraints:

1. $\mathbf{z}$ is **short**: $\|\mathbf{z}\|_{\infty} \leq \beta$ (coefficients are small, e.g., $\beta = 2^{17}$).
2. $\mathbf{z} \neq \mathbf{0}$.

Image generated by Deepseek LLM for conciseness of expression in equations (Deepseek, 2024), (National Institute of Standards and Technology, 2024). LaTeX on Microsoft Word was providing difficulties in implementing the correct expression format.

Within the Digital Signature Scheme, ML-DSA executes forty-nine algorithmic subroutines to generate a robust digital signature which serves to reinforce the foundational cryptographic aspects of non-repudiation within the context of data exchange, or the value of integrity within the CIA data security triad. Like its accompanying NIST PQC-standardized algorithm FIPS 203, ML-DSA can be implemented with three degrees of increasing signature complexity and decreasing efficiency, ML-DSA 44, ML-DSA 65, and ML-DSA 87.

We'll go ahead and implement a testing of the FIPS-204 ML-DSA utilizing our Kali-Linux machine and the Liboqs library.

We'll be generating a block of C code, utilizing Nano as our text processing environment, to implement the ML-DSA Digital Signature Scheme. Like our previous exploration of FIPS-203, the baseline functionality of the code will be developed in tandem with Claude 3.7 Sonnet with my additional oversight of testing and checking for code deprecation. In this execution, Claude 3.7 again pulls most of its methodology and call structure from the Open Quantum Safe project (OQS, 2023).

Creating `dilithium_test.c` in nano and adding the appropriate C code.



```
kali@kali: ~/pqc-research/experiments/pqc_testing/signatures
File Actions Edit View Help
GNU nano 8.3 dilithium_test.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <oqs/oqs.h>

int main(void) {
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *signature = NULL;
    uint8_t message[100] = "This is a test message to be signed with Dilithium";
    size_t message_len = strlen((char*)message);
    size_t signature_len;
    OQS_STATUS ret = OQS_ERROR;

    // Initialize the Dilithium algorithm (medium security level)
    OQS_SIG *sig = OQS_SIG_new(OQS_SIG_alg_dilithium_3);
    if (sig == NULL) {
        printf("Error: couldn't initialize Dilithium-3\n");
        return EXIT_FAILURE;
    }

    printf("Successfully initialized Dilithium-3\n");
    printf("Public key length: %zu bytes\n", sig->length_public_key);
    printf("Secret key length: %zu bytes\n", sig->length_secret_key);
    printf("Maximum signature length: %zu bytes\n", sig->length_signature);

    // Allocate memory
    public_key = malloc(sig->length_public_key);
    secret_key = malloc(sig->length_secret_key);
    signature = malloc(sig->length_signature);

    if ((public_key == NULL) || (secret_key == NULL) || (signature == NULL)) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
kali@kali: ~/pqc-research/experiments/pqc_testing/signatures
File Actions Edit View Help
GNU nano 8.3 dilithium_test.c *
if ((public_key = NULL) || (secret_key = NULL) || (signature = NULL)) {
    printf("Error: malloc failed\n");
    ret = OQS_ERROR;
    goto err;
}

// Generate keypair
ret = OQS_SIG_keypair(sig, public_key, secret_key);
if (ret != OQS_SUCCESS) {
    printf("Error: keypair generation failed\n");
    goto err;
}

// Sign the message
ret = OQS_SIG_sign(sig, signature, &signature_len, message, message_len, secret_key);
if (ret != OQS_SUCCESS) {
    printf("Error: signing failed\n");
    goto err;
}
printf("Signature created. Length: %zu bytes\n", signature_len);

// Verify the signature
ret = OQS_SIG_verify(sig, message, message_len, signature, signature_len, public_key);
if (ret != OQS_SUCCESS) {
    printf("Error: verification failed\n");
    goto err;
}

printf("Success! Signature verified correctly.\n");

// Test signature verification with modified message
printf("Testing with modified message (should fail verification)...\n");

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
kali@kali: ~/pqc-research/experiments/pqc_testing/signatures
File Actions Edit View Help
GNU nano 8.3 dilithium_test.c *
// Verify the signature
ret = OQS_SIG_verify(sig, message, message_len, signature, signature_len, public_key);
if (ret != OQS_SUCCESS) {
    printf("Error: verification failed\n");
    goto err;
}

printf("Success! Signature verified correctly.\n");

// Test signature verification with modified message
printf("Testing with modified message (should fail verification)...\n");
message[0] = 'X'; // Modify the first character
ret = OQS_SIG_verify(sig, message, message_len, signature, signature_len, public_key);
if (ret != OQS_SUCCESS) {
    printf("Verification correctly failed for modified message.\n");
    ret = OQS_SUCCESS; // This is expected behavior
} else {
    printf("Error: Verification succeeded for modified message!\n");
    ret = OQS_ERROR;
    goto err;
}

err:
// Clean up
if (public_key) free(public_key);
if (secret_key) free(secret_key);
if (signature) free(signature);
OQS_SIG_free(sig);

return (ret == OQS_SUCCESS) ? EXIT_SUCCESS : EXIT_FAILURE;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Adding the correct pathways to our compile.sh



```
kali@kali: ~/pqc-research/experiments/pqc_testing/signatures
GNU nano 8.3                                compile.sh *
#!/bin/bash

# Detect the liboqs directory
LIBOQS_DIR=~/.pqc-research/tools/liboqs

# Compile Kyber test
gcc -o pqc_testing/key_exchange/kyber_test pqc_testing/key_exchange/kyber_test.c \
    -I${LIBOQS_DIR}/build/include -L${LIBOQS_DIR}/build/lib -loqs -lcrypto -ldl

echo "Compilation complete. Set the library path before running:"
echo "export LD_LIBRARY_PATH=\$LD_LIBRARY_PATH:${LIBOQS_DIR}/build/lib"

# Compile Dilithium test
gcc -o pqc_testing/signatures/dilithium_test pqc_testing/signatures/dilithium_test.c \
    -I${LIBOQS_DIR}/build/include -L${LIBOQS_DIR}/build/lib -loqs -lcrypto -ldl

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

(Anthropic, 2025) and (Open Quantum Safe, 2023).

Running the dilithium_test in the terminal.

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
benchmarks key_exchange signatures
(kali@kali)-[~/pqc-research/experiments/pqc_testing]
$ cd signatures
(kali@kali)-[~/pqc-research/experiments/pqc_testing/signatures]
$ ls
(kali@kali)-[~/pqc-research/experiments/pqc_testing/signatures]
$ nano dilithium_test.c
(kali@kali)-[~/pqc-research/experiments/pqc_testing/signatures]
$ cd ~/pqc-research/experiments
nano compile.sh
(kali@kali)-[~/pqc-research/experiments]
$ ./compile.sh
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/pqc-research/tools/liboqs/build/lib
./pqc_testing/signatures/dilithium_test
Compilation complete. Set the library path before running:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/kali/pqc-research/tools/liboqs/build/lib
Successfully initialized Dilithium-3
Public key length: 1952 bytes
Secret key length: 4000 bytes
Maximum signature length: 3293 bytes
Signature created. Length: 3293 bytes
Success! Signature verified correctly.
Testing with modified message (should fail verification)...
Verification correctly failed for modified message.
(kali@kali)-[~/pqc-research/experiments]
$ echo Zach Bradley
Zach Bradley
(kali@kali)-[~/pqc-research/experiments]
$
```

This successful test of the ML-DSA (CRYSTALS-dilithium) Digital Signature Scheme provides:

A Public Key of 1952 bytes.

A Secret Key Length of 4000 bytes

A Maximum Signature Length of 3292 bytes.

A modified message was generated to test the integrity of the digital signature and failed as expected.

Contextualizing Quantum Resistant Encryption and Potential Defenses

Security Considerations within the Quantum Environment: FIPS 205

The third type of post-quantum cryptographic protocol made available to the public via NIST's standardization project is FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA). This standard is based upon the SPHINCS+ submission to the Post-Quantum Cryptography Standardization project. The algorithmic construction of FIPS 205 involves a combination of hash-based primitives, such as Merkle Trees and FTS (few time signatures) to provide a digital signature standard, where reliance on previous iterations of the hash state are not required (National Institute of Standards and Technology, 2025). This provides additional ease of use and efficient key management applications.

Considering the desires of the National Institute of Standards and Technology to implement a structured defense in depth across the post-quantum cryptographic standards, FIPS 205 is considered to be a backup digital signature algorithm which relies on alternative mathematical principles to hedge against potential attacks in the event that ML-DSA has been compromised. In addition to this, the question of performance and efficiency across platforms has positioned ML-DSA as the frontrunner for digital signature generation in devices and network structures that require smaller signature sizes and more efficient computation times, for instance it is suggested that ML-DSA would more efficiently serve these purposes within network containing IoT devices and involving TLS handshakes, while the larger hash-based signatures provided by SLH-DSA are more useful in systems which require less low-latency performance and have the luxury of operating with larger bandwidth (Callan, 2024).

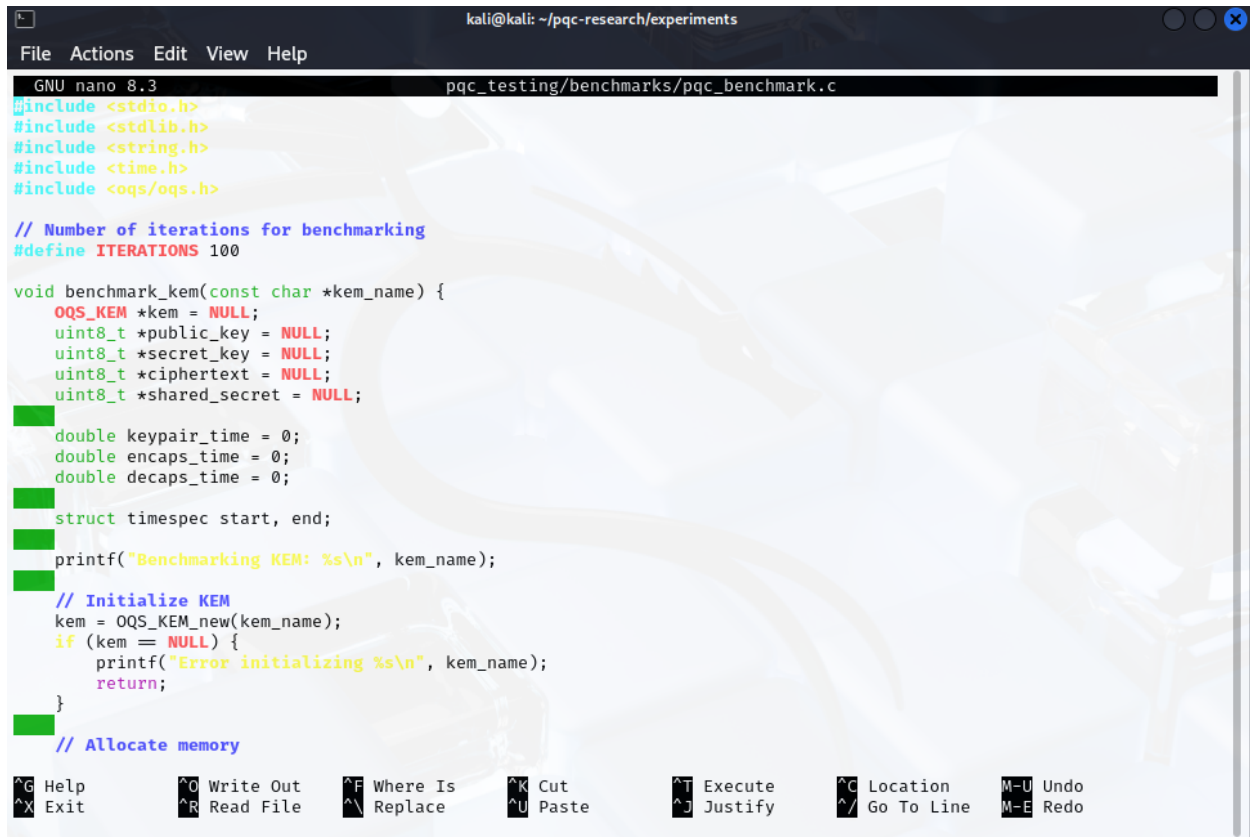
In terms of the mathematical functions involved with the implementation of SLH-DSA, the National Institute of Standards and Technology has stated an overview of the methodologies involved, "SLH-DSA is a stateless hash-based signature scheme that is constructed using other hash-based signature schemes as components: (1) a few-time signature scheme, forest of random subsets (FORS), and (2) a multi-time signature scheme, the eXtended Merkle Signature Scheme (XMSS). XMSS is constructed using the hash-based one-time signature scheme Winternitz One-Time 2 2The WOTS+ and XMSS schemes that are used as components of SLH-DSA are not the same as the WOTS+ and XMSS schemes in RFC 8391 [11] and SP 800-208 [12]. , 264, 266 Signature Plus (WOTS+) as a component (National Institute of Standards and Technology, 2024). Folded into an execution of 25 algorithmic subroutines, SLH-DSA relies on two varying hash formats, SHA-256 and SHAKE 128/256 for pseudorandom key generation, Merkle Tree node hashing, and few time signature (FTS) keys.

Since FIPS 205 SLH-DSA is considered to be a backup digital signature algorithm for FIPS 204 ML-DSA, the key size and processing speed considerations are less restrictive, and allow for a more conservative placement of the SLH-DSA within an organizations data encryption infrastructure. As such, I have decided to include the processing of FIPS 205 into a

benchmark testing implementation, instead of running a stand-alone key generation for the standard at this time.

At this point, let's go ahead and generate some code that provides some benchmark timing data for both the implementation of FIPS 203 ML-KEM, FIPS 204 ML-DSA, and some of the other algorithms.

We'll use Claude 3.7 Sonnet to generate some more C code, edit the code in the Nano text editor to chase down/remove/and edit the deprecation that occurs between the current Liboqs library and Claude's last update.



```
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <oqs/oqs.h>

// Number of iterations for benchmarking
#define ITERATIONS 100

void benchmark_kem(const char *kem_name) {
    OQS_KEM *kem = NULL;
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *ciphertext = NULL;
    uint8_t *shared_secret = NULL;

    double keypair_time = 0;
    double encaps_time = 0;
    double decaps_time = 0;

    struct timespec start, end;

    printf("Benchmarking KEM: %s\n", kem_name);

    // Initialize KEM
    kem = OQS_KEM_new(kem_name);
    if (kem == NULL) {
        printf("Error initializing %s\n", kem_name);
        return;
    }

    // Allocate memory
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c

// Allocate memory
public_key = malloc(kem->length_public_key);
secret_key = malloc(kem->length_secret_key);
ciphertext = malloc(kem->length_ciphertext);
shared_secret = malloc(kem->length_shared_secret);

if (!public_key || !secret_key || !ciphertext || !shared_secret) {
    printf("Memory allocation error\n");
    goto cleanup;
}

// Key generation benchmark
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < ITERATIONS; i++) {
    OQS_KEM_keypair(kem, public_key, secret_key);
}
clock_gettime(CLOCK_MONOTONIC, &end);
keypair_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

// Encapsulation benchmark
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < ITERATIONS; i++) {
    OQS_KEM_encaps(kem, ciphertext, shared_secret, public_key);
}
clock_gettime(CLOCK_MONOTONIC, &end);
encaps_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

// Decapsulation benchmark
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < ITERATIONS; i++) {
    OQS_KEM_decaps(kem, shared_secret, ciphertext, secret_key);
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c

// Decapsulation benchmark
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < ITERATIONS; i++) {
    OQS_KEM_decaps(kem, shared_secret, ciphertext, secret_key);
}
clock_gettime(CLOCK_MONOTONIC, &end);
decaps_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

// Results
printf("Results for %s (%d iterations):\n", kem_name, ITERATIONS);
printf("  Key generation: %.6f ms/op\n", (keypair_time * 1000) / ITERATIONS);
printf("  Encapsulation: %.6f ms/op\n", (encaps_time * 1000) / ITERATIONS);
printf("  Decapsulation: %.6f ms/op\n", (decaps_time * 1000) / ITERATIONS);
printf("  Public key: %zu bytes\n", kem->length_public_key);
printf("  Secret key: %zu bytes\n", kem->length_secret_key);
printf("  Ciphertext: %zu bytes\n", kem->length_ciphertext);
printf("  Shared secret: %zu bytes\n", kem->length_shared_secret);

cleanup:
// Free allocated memory
if (public_key) free(public_key);
if (secret_key) free(secret_key);
if (ciphertext) free(ciphertext);
if (shared_secret) free(shared_secret);
OQS_KEM_free(kem);
}

void benchmark_sig(const char *sig_name) {
    OQS_SIG *sig = NULL;
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *signature = NULL;
}
```

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c
void benchmark_sig(const char *sig_name) {
    OQS_SIG *sig = NULL;
    uint8_t *public_key = NULL;
    uint8_t *secret_key = NULL;
    uint8_t *signature = NULL;
    uint8_t message[50] = "This is a test message for signature benchmarking";
    size_t message_len = strlen((char*)message);
    size_t signature_len;

    double keypair_time = 0;
    double sign_time = 0;
    double verify_time = 0;

    struct timespec start, end;

    printf("Benchmarking Signature: %s\n", sig_name);

    // Initialize Signature
    sig = OQS_SIG_new(sig_name);
    if (sig == NULL) {
        printf("Error initializing %s\n", sig_name);
        return;
    }

    // Allocate memory
    public_key = malloc(sig->length_public_key);
    secret_key = malloc(sig->length_secret_key);
    signature = malloc(sig->length_signature);

    if (!public_key || !secret_key || !signature) {
        printf("Memory allocation error\n");
        goto cleanup;
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo

```

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c
    if (!public_key || !secret_key || !signature) {
        printf("Memory allocation error\n");
        goto cleanup;
    }

    // Key generation benchmark
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < ITERATIONS; i++) {
        OQS_SIG_keypair(sig, public_key, secret_key);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    keypair_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Generate a key pair for signing and verification
    OQS_SIG_keypair(sig, public_key, secret_key);

    // Signing benchmark
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < ITERATIONS; i++) {
        OQS_SIG_sign(sig, signature, &signature_len, message, message_len, secret_key);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    sign_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    // Verification benchmark
    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < ITERATIONS; i++) {
        OQS_SIG_verify(sig, message, message_len, signature, signature_len, public_key);
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    verify_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo

```

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c

// Verification benchmark
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < ITERATIONS; i++) {
    OQS_SIG_verify(sig, message, message_len, signature, signature_len, public_key);
}
clock_gettime(CLOCK_MONOTONIC, &end);
verify_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

// Results
printf("Results for %s (%d iterations):\n", sig_name, ITERATIONS);
printf("  Key generation: %.6f ms/op\n", (keypair_time * 1000) / ITERATIONS);
printf("  Signing: %.6f ms/op\n", (sign_time * 1000) / ITERATIONS);
printf("  Verification: %.6f ms/op\n", (verify_time * 1000) / ITERATIONS);
printf("  Public key: %zu bytes\n", sig->length_public_key);
printf("  Secret key: %zu bytes\n", sig->length_secret_key);
printf("  Signature: %zu bytes\n", signature_len);

cleanup:
// Free allocated memory
if (public_key) free(public_key);
if (secret_key) free(secret_key);
if (signature) free(signature);
OQS_SIG_free(sig);
}

int main() {
    // Initialize OQS
    OQS_init();

    printf("=== Post-Quantum Cryptography Benchmark Suite ===\n");
    printf("NIST Selected and Alternate Algorithms\n");

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^I Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 pqc_testing/benchmarks/pqc_benchmark.c

int main() {
    // Initialize OQS
    OQS_init();

    printf("=== Post-Quantum Cryptography Benchmark Suite ===\n");
    printf("NIST Selected and Alternate Algorithms\n");

    // Run benchmarks for selected KEMs
    printf("=== Key Encapsulation Mechanisms (KEMs) ===\n");
    benchmark_kem(OQS_KEM_alg_kyber_512);
    benchmark_kem(OQS_KEM_alg_kyber_768);
    benchmark_kem(OQS_KEM_alg_kyber_1024);

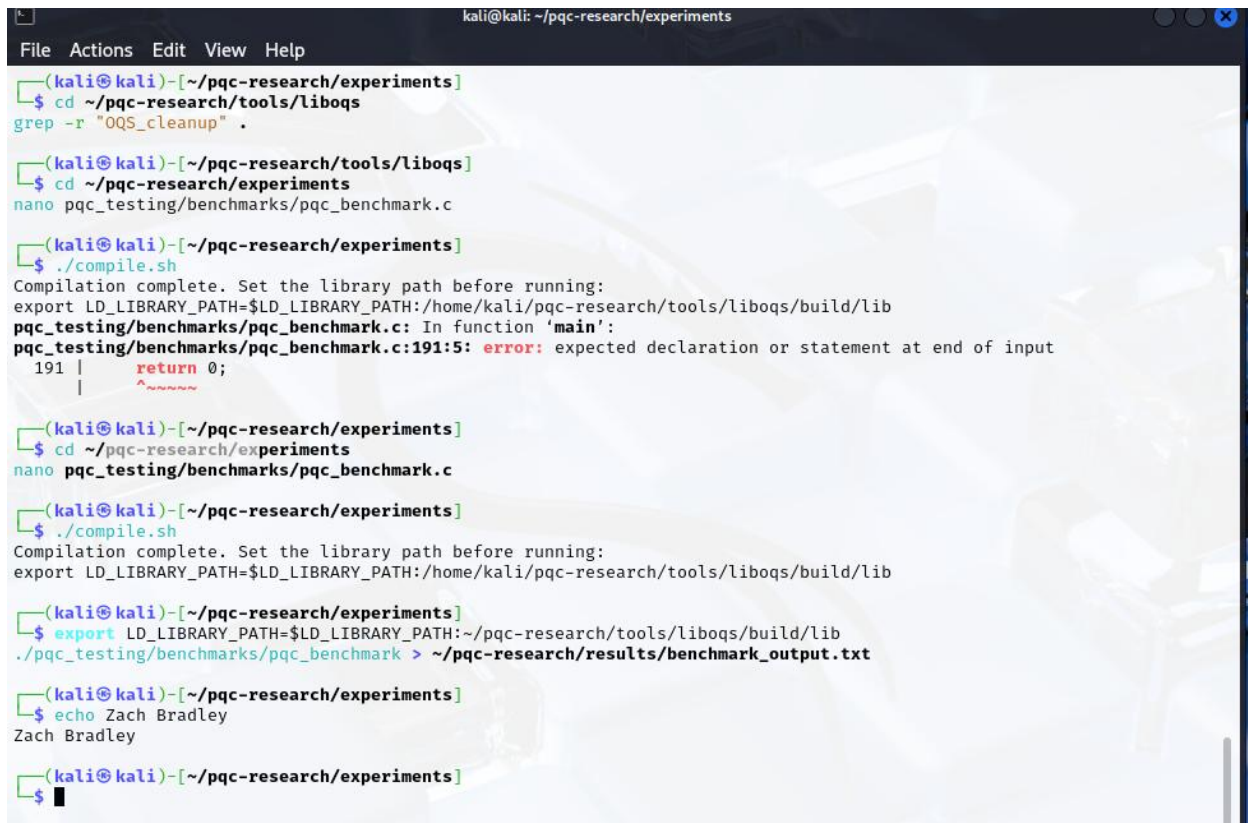
    // Try to run some alternative KEMs if available
    benchmark_kem(OQS_KEM_alg_hqc_128);
    benchmark_kem(OQS_KEM_alg_classic_mceliece_348864);

    // Run benchmarks for selected signature algorithms
    printf("\n=== Digital Signature Algorithms ===\n");
    benchmark_sig(OQS_SIG_alg_dilithium_2);
    benchmark_sig(OQS_SIG_alg_dilithium_3);
    benchmark_sig(OQS_SIG_alg_dilithium_5);
    benchmark_sig(OQS_SIG_alg_falcon_512);
    benchmark_sig(OQS_SIG_alg_falcon_1024);
    benchmark_sig(OQS_SIG_alg_sphincs_sha2_128f_simple);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^I Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```


The obligatory deprecation chase and re-write.



```
kali@kali: ~/pqc-research/experiments
File Actions Edit View Help

(kali@kali)~[~/pqc-research/experiments]
$ cd ~/pqc-research/tools/liboqs
grep -r "OQS_cleanup" .

(kali@kali)~[~/pqc-research/tools/liboqs]
$ cd ~/pqc-research/experiments
nano pqc_testing/benchmarks/pqc_benchmark.c

(kali@kali)~[~/pqc-research/experiments]
$ ./compile.sh
Compilation complete. Set the library path before running:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/kali/pqc-research/tools/liboqs/build/lib
pqc_testing/benchmarks/pqc_benchmark.c: In function 'main':
pqc_testing/benchmarks/pqc_benchmark.c:191:5: error: expected declaration or statement at end of input
191 |     return 0;
    |

(kali@kali)~[~/pqc-research/experiments]
$ cd ~/pqc-research/experiments
nano pqc_testing/benchmarks/pqc_benchmark.c

(kali@kali)~[~/pqc-research/experiments]
$ ./compile.sh
Compilation complete. Set the library path before running:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/kali/pqc-research/tools/liboqs/build/lib

(kali@kali)~[~/pqc-research/experiments]
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:~/pqc-research/tools/liboqs/build/lib
./pqc_testing/benchmarks/pqc_benchmark > ~/pqc-research/results/benchmark_output.txt

(kali@kali)~[~/pqc-research/experiments]
$ echo Zach Bradley
Zach Bradley

(kali@kali)~[~/pqc-research/experiments]
$
```

Here are the results of our benchmarking program on my 2024 Kali Linux Virtual Machine, which operates on VMware Workstation.

Virtual Machine specifications: Kali Rolling (2024.4) x64 2024-11-30. 4 Processors allocated. 8.6 GB of memory. 80.1 GB of hard disk space.

My host device is a Lenovo Thinkpad with AMD Ryzen 7 PRO 6850U with Radeon Graphics 2.70 GHz Processor with 16.0 GB (14.8 GB usable) of installed RAM.

This is a 64-bit operating system, x64-based processor.

=== Post-Quantum Cryptography Benchmark Suite ===

NIST Selected and Alternate Algorithms

=== Key Encapsulation Mechanisms (KEMs) ===

Benchmarking KEM: Kyber512

Results for Kyber512 (100 iterations):

Key generation: 0.048446 ms/op
Encapsulation: 0.034312 ms/op
Decapsulation: 0.040858 ms/op
Public key: 800 bytes
Secret key: 1632 bytes
Ciphertext: 768 bytes
Shared secret: 32 bytes

Benchmarking KEM: Kyber768

Results for Kyber768 (100 iterations):

Key generation: 0.043734 ms/op
Encapsulation: 0.050532 ms/op
Decapsulation: 0.058814 ms/op
Public key: 1184 bytes
Secret key: 2400 bytes
Ciphertext: 1088 bytes
Shared secret: 32 bytes

Benchmarking KEM: Kyber1024

Results for Kyber1024 (100 iterations):

Key generation: 0.066389 ms/op
Encapsulation: 0.073627 ms/op
Decapsulation: 0.083742 ms/op
Public key: 1568 bytes
Secret key: 3168 bytes
Ciphertext: 1568 bytes
Shared secret: 32 bytes

Benchmarking KEM: HQC-128

Results for HQC-128 (100 iterations):

Key generation: 1.301007 ms/op
Encapsulation: 2.684246 ms/op
Decapsulation: 4.268673 ms/op
Public key: 2249 bytes
Secret key: 2305 bytes
Ciphertext: 4433 bytes
Shared secret: 64 bytes

=== Digital Signature Algorithms ===

Benchmarking Signature: Dilithium2

Results for Dilithium2 (100 iterations):

■ Key generation: 0.066124 ms/op
Signing: 0.313506 ms/op
Verification: 0.072322 ms/op
Public key: 1312 bytes
Secret key: 2528 bytes
Signature: 2420 bytes

Benchmarking Signature: Dilithium3

Results for Dilithium3 (100 iterations):

Key generation: 0.119823 ms/op
Signing: 0.565369 ms/op
Verification: 0.117627 ms/op
Public key: 1952 bytes
Secret key: 4000 bytes
Signature: 3293 bytes

Benchmarking Signature: Dilithium5

Results for Dilithium5 (100 iterations):

Key generation: 0.177939 ms/op
Signing: 0.552473 ms/op
Verification: 0.185942 ms/op
Public key: 2592 bytes
Secret key: 4864 bytes
Signature: 4595 bytes

Benchmarking Signature: Falcon-512

Results for Falcon-512 (100 iterations):

Key generation: 10.525279 ms/op
Signing: 3.043487 ms/op
Verification: 0.029202 ms/op
Public key: 897 bytes
Secret key: 1281 bytes
Signature: 656 bytes

Benchmarking Signature: Falcon-1024

Results for Falcon-1024 (100 iterations):

Key generation: 30.483769 ms/op
Signing: 6.731821 ms/op
Verification: 0.059054 ms/op
Public key: 1793 bytes
Secret key: 2305 bytes

```
Secret key: 1281 bytes
Signature: 656 bytes
Benchmarking Signature: Falcon-1024
Results for Falcon-1024 (100 iterations):
Key generation: 30.483769 ms/op
■ Signing: 6.731821 ms/op
Verification: 0.059054 ms/op
Public key: 1793 bytes
Secret key: 2305 bytes
Signature: 1271 bytes
Benchmarking Signature: SPHINCS+-SHA2-128f-simple
Results for SPHINCS+-SHA2-128f-simple (100 iterations):
Key generation: 1.213616 ms/op
Signing: 27.303938 ms/op
Verification: 1.595924 ms/op
Public key: 32 bytes
Secret key: 64 bytes
Signature: 17088 bytes
~
~
~
~
~
~
~
~
```

Now we will use python libraries to visualize the results of our benchmark testing.

We will use Claude 3.7 Sonnet with Python3, matplotlib, and numpy to create a graphic size and speed comparison of the various ML-KEM and ML-DSA protocols available in Liboqs and process said protocols on my local Kali Linux machine.


```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 analyze_results.py
import os
import matplotlib.pyplot as plt
import numpy as np
import re
import sys

def parse_benchmark_results(filename):
    with open(filename, 'r') as f:
        content = f.read()

    # Split content into KEM and SIG sections
    kem_section = re.search(r'=== Key Encapsulation Mechanisms \\\(KEMs\\) ===\s*(.*?)(?===)', content, re.DOTALL)
    sig_section = re.search(r'=== Digital Signature Algorithms ===\s*(.*)', content, re.DOTALL)

    kem_results = {}
    sig_results = {}

    if kem_section:
        kem_content = kem_section.group(1)
        kem_blocks = re.findall(r'Results for (.*?) \\\(\\d+ iterations\\):(.*?)\\(=Benchmarking!$\\)', kem_content, re.DOTALL)

        for kem_name, block in kem_blocks:
            kem_name = kem_name.strip()

            # Extract metrics
            keygen_time = float(re.search(r'Key generation: ([\\d.]+) ms/op', block).group(1)) if re.search(r'Key gene>
encaps_time = float(re.search(r'Encapsulation: ([\\d.]+) ms/op', block).group(1)) if re.search(r'Encapsula>
decaps_time = float(re.search(r'Decapsulation: ([\\d.]+) ms/op', block).group(1)) if re.search(r'Decapsula>
pub_key_size = int(re.search(r'Public key: (\\d+) bytes', block).group(1)) if re.search(r'Public key: (\\d+>
sec_key_size = int(re.search(r'Secret key: (\\d+) bytes', block).group(1)) if re.search(r'Secret key: (\\d+>
ciphertext_size = int(re.search(r'Ciphertext: (\\d+) bytes', block).group(1)) if re.search(r'Ciphertext: (>

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line M-E Redo
```

```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 analyze_results.py

return kem_results, sig_results

def plot_kem_performance(kem_results, output_dir):
    if not kem_results:
        print("No KEM results to plot")
        return

    # Extract data
    kem_names = list(kem_results.keys())
    keygen_times = [kem_results[name]['keygen_time'] for name in kem_names]
    encaps_times = [kem_results[name]['encaps_time'] for name in kem_names]
    decaps_times = [kem_results[name]['decaps_time'] for name in kem_names]

    # Create performance chart
    fig, ax = plt.subplots(figsize=(12, 6))
    x = np.arange(len(kem_names))
    width = 0.25

    ax.bar(x - width, keygen_times, width, label='Key Generation')
    ax.bar(x, encaps_times, width, label='Encapsulation')
    ax.bar(x + width, decaps_times, width, label='Decapsulation')

    ax.set_yscale('log')
    ax.set_ylabel('Time (ms/op) - log scale')
    ax.set_title('KEM Performance Comparison')
    ax.set_xticks(x)
    ax.set_xticklabels(kem_names, rotation=45, ha='right')
    ax.legend()
    plt.tight_layout()
    plt.savefig(f'{output_dir}/kem_performance.png')
    plt.close()
```

```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 analyze_results.py
# Create size chart
pub_key_sizes = [kem_results[name]['pub_key_size'] for name in kem_names]
sec_key_sizes = [kem_results[name]['sec_key_size'] for name in kem_names]
ciphertext_sizes = [kem_results[name]['ciphertext_size'] for name in kem_names]

fig, ax = plt.subplots(figsize=(12, 6))

ax.bar(x - width, pub_key_sizes, width, label='Public Key')
ax.bar(x, sec_key_sizes, width, label='Secret Key')
ax.bar(x + width, ciphertext_sizes, width, label='Ciphertext')

ax.set_yscale('log')
ax.set_ylabel('Size (bytes) - log scale')
ax.set_title('KEM Size Comparison')
ax.set_xticks(x)
ax.set_xticklabels(kem_names, rotation=45, ha='right')
ax.legend()
plt.tight_layout()
plt.savefig(f"{output_dir}/kem_sizes.png")
plt.close()

def plot_sig_performance(sig_results, output_dir):
    if not sig_results:
        print("No signature results to plot")
        return

    # Extract data
    sig_names = list(sig_results.keys())
    keygen_times = [sig_results[name]['keygen_time'] for name in sig_names]
    sign_times = [sig_results[name]['sign_time'] for name in sig_names]
    verify_times = [sig_results[name]['verify_time'] for name in sig_names]

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 analyze_results.py

return kem_results, sig_results

def plot_kem_performance(kem_results, output_dir):
    if not kem_results:
        print("No KEM results to plot")
        return

    # Extract data
    kem_names = list(kem_results.keys())
    keygen_times = [kem_results[name]['keygen_time'] for name in kem_names]
    encaps_times = [kem_results[name]['encaps_time'] for name in kem_names]
    decaps_times = [kem_results[name]['decaps_time'] for name in kem_names]

    # Create performance chart
    fig, ax = plt.subplots(figsize=(12, 6))
    x = np.arange(len(kem_names))
    width = 0.25

    ax.bar(x - width, keygen_times, width, label='Key Generation')
    ax.bar(x, encaps_times, width, label='Encapsulation')
    ax.bar(x + width, decaps_times, width, label='Decapsulation')

    ax.set_yscale('log')
    ax.set_ylabel('Time (ms/op) - log scale')
    ax.set_title('KEM Performance Comparison')
    ax.set_xticks(x)
    ax.set_xticklabels(kem_names, rotation=45, ha='right')
    ax.legend()
    plt.tight_layout()
    plt.savefig(f"{output_dir}/kem_performance.png")
    plt.close()

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help
GNU nano 8.3 analyze_results.py

ax.set_yscale('log')
ax.set_ylabel('Size (bytes) - log scale')
ax.set_title('Signature Size Comparison')
ax.set_xticks(x)
ax.set_xticklabels(sig_names, rotation=45, ha='right')
ax.legend()
plt.tight_layout()
plt.savefig(f"{output_dir}/sig_sizes.png")
plt.close()

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python analyze_results.py <benchmark_output_file>")
        sys.exit(1)

    output_dir = "../results"

    # Create the output directory if it doesn't exist
    os.makedirs(output_dir, exist_ok=True)

    kem_results, sig_results = parse_benchmark_results(sys.argv[1])

    plot_kem_performance(kem_results, output_dir)
    plot_sig_performance(sig_results, output_dir)

    print("Analysis complete. Results saved to:")
    print(f"    {output_dir}/kem_performance.png")
    print(f"    {output_dir}/kem_sizes.png")
    print(f"    {output_dir}/sig_performance.png")
    print(f"    {output_dir}/sig_sizes.png")

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

(Anthropic, 2025) and (Open Quantum Safe, 2023)

There were some issues with the python script in creating a proper directory for the storage of the graphical results. I had to go back and add the # Create the output directory if it doesn't exist # section and "import os" at top of the script as an additional import.

Here is an example of some of my errors and recovery in the command line.

```
(pqc-venv)kali@kali: ~/pqc-research/experiments
File Actions Edit View Help

self._print_pil(filename_or_obj, "png", pil_kwargs, metadata)
~~~~~
File "/home/kali/pqc-research/pqc-venv/lib/python3.13/site-packages/matplotlib/backends/backend_agg.py", line 430,
in _print_pil
    mpl.image.imsave(
        ^
        filename_or_obj, self.buffer_rgba(), format=fmt, origin="upper",
        ~~~~~
        dpi=self.figure.dpi, metadata=metadata, pil_kwargs=pil_kwargs)
        ~~~~~
File "/home/kali/pqc-research/pqc-venv/lib/python3.13/site-packages/matplotlib/image.py", line 1644, in imsave
    image.save(fname, **pil_kwargs)
        ~~~~~
File "/home/kali/pqc-research/pqc-venv/lib/python3.13/site-packages/PIL/Image.py", line 2576, in save
    fp = builtins.open(filename, "w+b")
FileNotFoundError: [Errno 2] No such file or directory: '../results/kem_performance.png'

(pqc-venv)-(kali@kali)-[~/pqc-research/experiments]
$ cd ~/pqc-research/experiments/pqc_testing/benchmarks
nano analyze_results.py

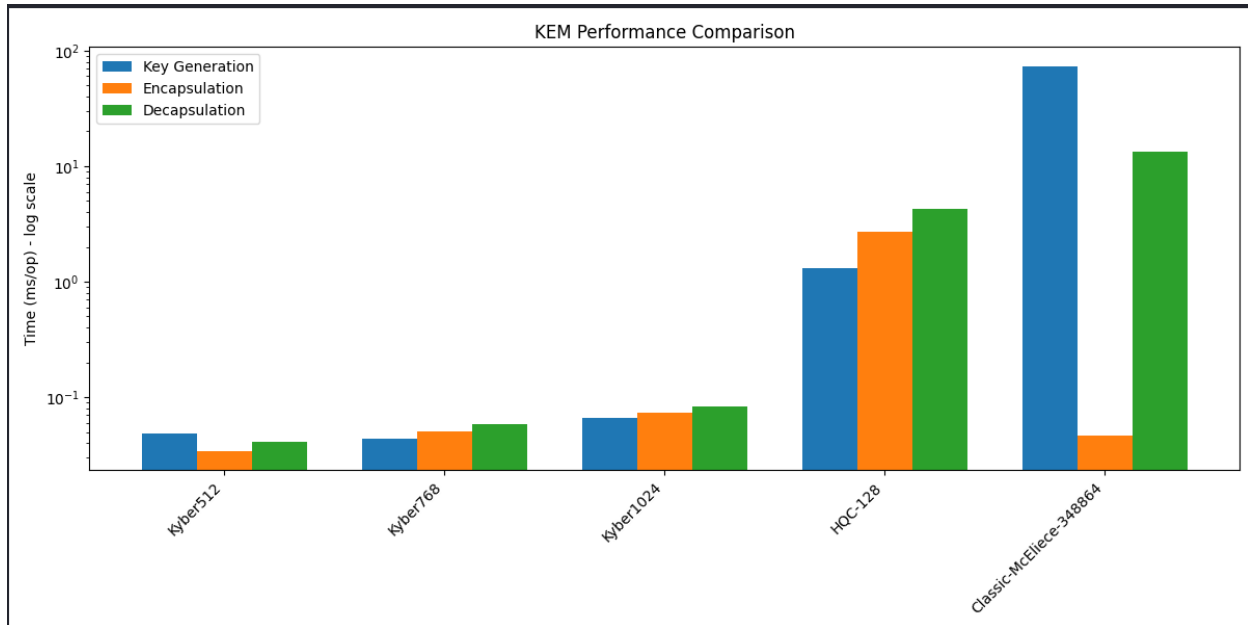
(pqc-venv)-(kali@kali)-[~/pqc-research/experiments/pqc_testing/benchmarks]
$ cd ~/pqc-research/experiments
python pqc_testing/benchmarks/analyze_results.py ~/pqc-research/results/benchmark_output.txt
Analysis complete. Results saved to:
  ../results/kem_performance.png
  ../results/kem_sizes.png
  ../results/sig_performance.png
  ../results/sig_sizes.png

(pqc-venv)-(kali@kali)-[~/pqc-research/experiments]
$ echo Zach Bradley
Zach Bradley

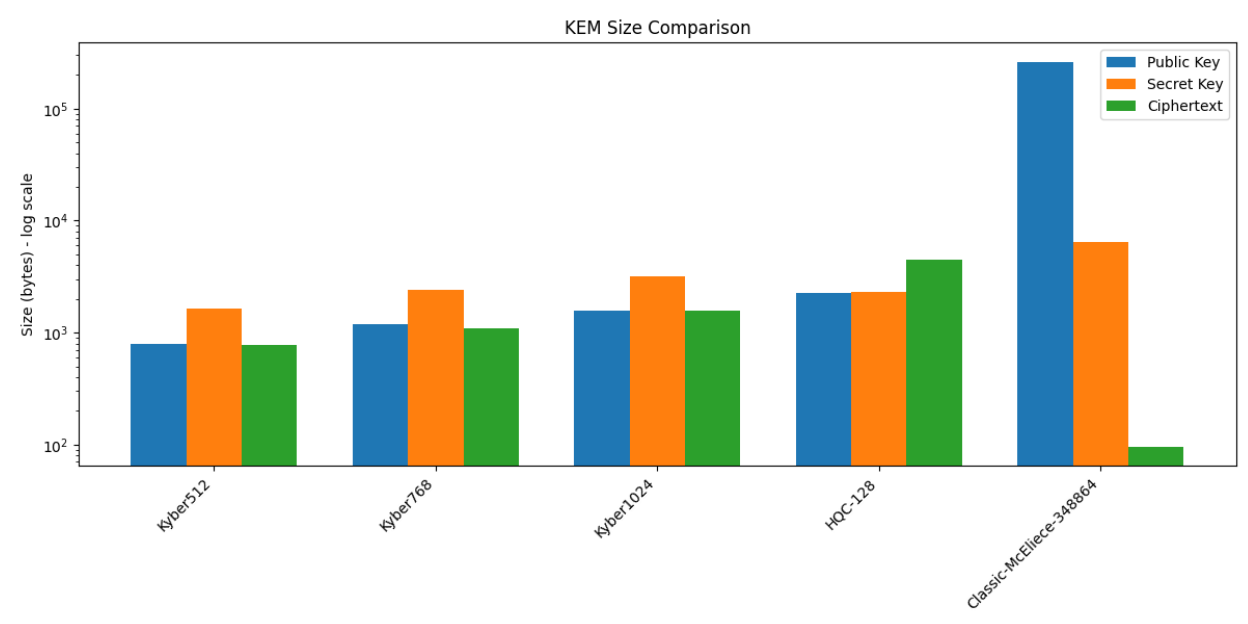
(pqc-venv)-(kali@kali)-[~/pqc-research/experiments]
$
```

Now let's go ahead and view the graphical representation of our benchmark testing on our Kali Linux VM.

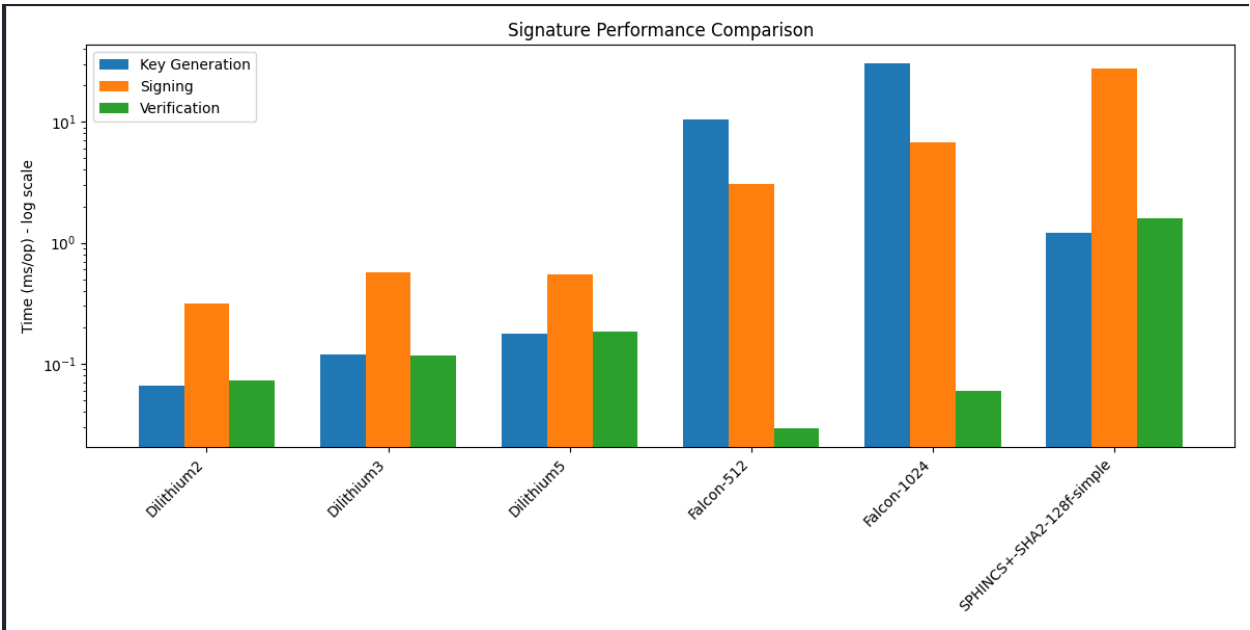
Key Encapsulation Mechanism Performance Benchmark Comparison.



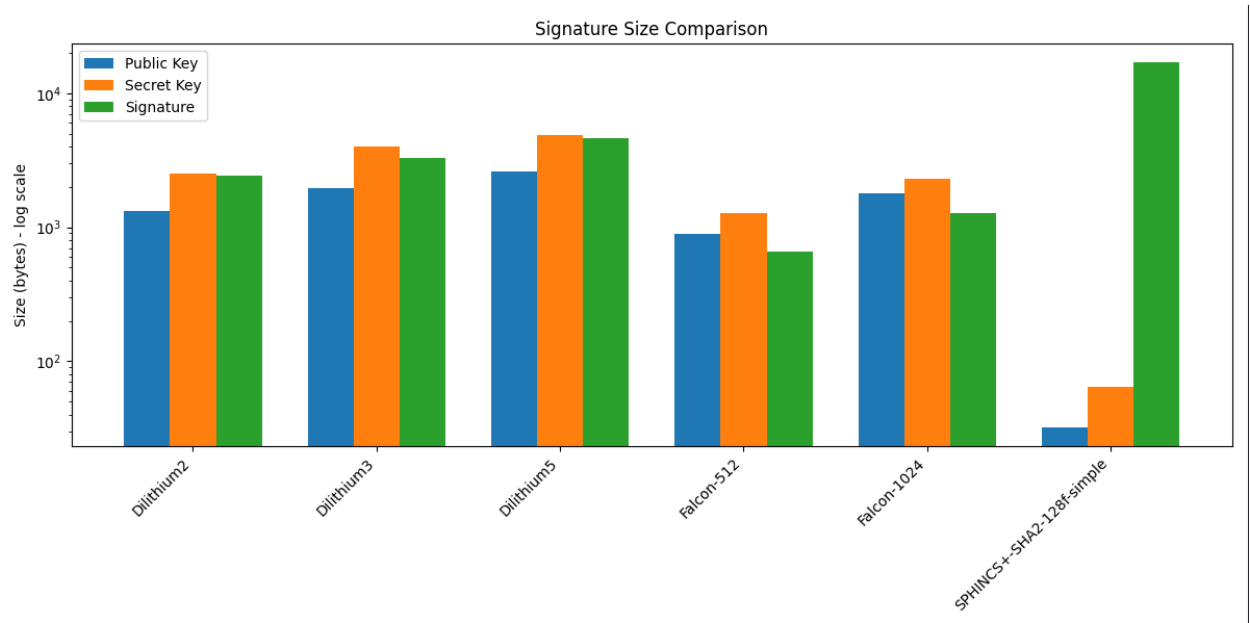
Key Encapsulation Mechanism Performance Size Comparison.



Digital Signature Performance Benchmark Comparison.



Digital Signature Size Benchmark Comparison



These graphical representations of our benchmark testing for FIPS ML-KEM, FIPS ML-DSA, and associated alternative library algorithms provide a visual explanation of the differences in processing time and size between both the various suggested NIST algorithms and the selected NIST algorithms. Basic benchmarking testing such as this exercise on Kali Linux VM, should be

replicated across various hardware, cross-referenced, and analyzed in order to provide a more complete display of the capabilities of these key encapsulation mechanisms, digital signature algorithms, and stateless hash-based digital signature algorithms along various platform channels. The relative discrepancy between performance indicators on various machines can be evaluated on a case-by-case basis to provide actionable insight for stakeholders interested in migrating their enterprise to post-quantum cryptographic protocols (National Institute of Standards and Technology, 2024).

Infrastructural Recommendations for Post-Quantum Encryption Algorithms:

The implementation of Quantum Resistant Protocols in Enterprise Infrastructure.

With the imminent arrival of quantum computers and quantum information systems looming on the near horizon, both government, enterprise, and private organization should immediately begin the transition to post-quantum encryption algorithms in order to safeguard their data from future threats. While the arrival of utility grade quantum computer's utilizing Shor's algorithm has yet to come, one methodology that threat actors are employing in their future quantum encryption breaking strategy is to presently collect and store the information they aim to decrypt in the future with quantum devices.

This "Store now, Decrypt later" (SNDL) approach leverages a vast data-collection strategy where potential threat-actors are currently collecting vast amounts of data for future potential quantum-decryption. Likewise, this "wide-net" strategy allows data security antagonists the opportunity to hedge their bets against the present encryption environment by saving data for future decryption. The potential of quantum decryption grants future access to untold infrastructural devastation, and the dark tide of currently encrypted data being siphoned off for later naked analysis is immense (Ivezic, 2025). When considering the implications of quantum decryption on the security of our geopolitical communication and intelligence infrastructure, it's clear that nation-state actors, such as Russia, China, and The United States are the most significant players participating in the bulk of SNDL data harvesting. The goal for this SNDL approach is to gather, consolidate, and eventually decrypt enough data to eavesdrop on secure communication channels; when persistent access and full eavesdropping capabilities are not enough, involvement in the disruption and manipulation of these geopolitical communication channels is the mature strategic offensive goal of the Nation-State SNDL strategy, examples would include forging defensive communications or decrypting decades worth of archived diplomatic cables for the purposes of targeted leaks and blackmail.

In addition to the various nation-state strategic approaches associated with the SNDL strategy, it is assumed that a significant number of independent, private threat actors are participating in the same general SNDL strategy. The difference between the private operators and the state-sanctioned data harvesters is a slight shifting in the target demographic. While nation-states may be targeting the defense and intelligence infrastructure of other nation states, private threat actors may be harvesting the encrypted data as it pertains to financial institutions,

corporate communications, business intelligence, and the infrastructure of blockchain technology. Blockchain technology in particular is vulnerable to the decryption of ECC and ECDSA, which provides the cryptographic structure of both Bitcoin and Ethereum wallets. Given sufficient quantum computing advancement, these wallets can be accessed retroactively when the encryption collected now through SNDL is cracked. Conservative estimates suggest that up to \$10 Billion in Bitcoin is currently vulnerable due to the reuse of ECDSA keys alone and emphasizes a need for updated cryptographic practices (Aggarwal et al., 2017).

Resistance to both strategic and rogue future quantum decryption threats are currently available in the present. The structured implementation of the post-quantum encryption algorithms released by the National Institute of Standards and Technology will provide a robust cryptographic infrastructure that is resistant to both classical and quantum decryption strategies for the foreseeable future.

The utilization of FIPS 203 ML-KEM as a public key infrastructure mechanism is vital to the secure exchange of symmetric keys in a post-quantum cryptographic infrastructure. For current actionable ease of use, increased processing speed, and fidelity to legacy encryption methodology, ML-KEM can be effectively folded into a current encryption infrastructure by using a hybrid approach featuring KEM as the public key encapsulation mechanism and the AES standard as the private, symmetric key in the infrastructure. This methodology leverages the security of ML-KEM against both quantum and classical decryption attacks, combined with the speed, security, and efficiency of AES for symmetric key exchange. I have shown throughout the course of this research project that FIPS 203 can be implemented on personal devices utilizing low amounts of disk space and RAM. This ease of use can be effectively transferred to an enterprise grade encryption infrastructure with very little processing and storage difference from legacy standards.

For digital certificate management and verification, the FIPS 204 ML-DSA standard provides a module-lattice based digital signature algorithm that secures the integrity and authenticity of data against classical and future quantum attacks. Government agencies, corporate data security practitioners, financial institutions, and private enterprise should begin the transition to utilizing FIPS 204 ML-DSA to generate digital signatures that are secure against future quantum threats.

In the event that organizations need particularly robust digital signature management, the National Institute of Standards and Technology has provided FIPS 205, Stateless Hash-Based Digital Signature Standard (SLH-DSA) as a secure backup digital signature standard. Institutional and enterprise practitioners should utilize this standard within their cryptographic infrastructure as a safeguard against the possible erosion of the security standards of ML-DSA. Although SLH-DSA has slower performance standards than ML-DSA, it provides greater mathematical resistance to quantum-enabled decryption attacks, and thus operates best as a contingency cryptographic protocol (National Institute of Standards and Technology, 2024).

From the perspective of the cybersecurity CIA triad, the National Institute of Standards and Technologies FIPS 203, 204, and 205 adhere to the recommendations in the following manner, FIPS 203 ML-KEM provides confidentiality to practitioners by providing a public/private key encapsulation mechanism that insulates against eavesdroppers who possess both classical and quantum decryption technologies.

The FIPS 204 ML-DSA and FIPS 205 SLH-DSA provide integrity to the communication channel by verifying the authenticity and origination of the data's sender. Availability is provided by all three FIPS protocols, as evidenced by the fact that all three algorithms can be executed on rudimentary local hardware, such as my Kali Linux virtual machine.

Implementing Encryption Infrastructure Management: The Technical Issues

Throughout the course of this project, I have explored the methodologies and implementations of basic, current, and publicly available quantum computing, the current execution capabilities of the Deutsch-Jozsa algorithm and Shor's algorithm, the National Institute of Standards and Technologies first-wave of post-quantum cryptographic standards FIPS 203, 204, 205 and detailed my progress with these new cryptographic and computational methods along the way.

For my exploration of current quantum computing platforms and availability, I utilized the Qiskit software development kit and the IBM Quantum Platform to interact locally with the Aer quantum simulator and remotely with IBM's quantum hardware. Apart from correctly importing Qiskit and its dependent libraries, setting up a Python virtual environment, and working with Jupyter notebooks to implement my Qiskit code executions on a stepwise basis, the majority of the technical work with the IBM Quantum platform has involved chasing down and altering deprecated code while executing jobs on quantum hardware. The entire IBM quantum platform is in the midst of a major transition, and a large majority of the IBM systems that I have used to complete my work will be moved and sunset by July 1st, 2025.

Since the Qiskit SDK and the IBM Quantum Platform have been experiencing such rapid expansion, development, and overhaul, oftentimes the code recommended in textbooks, online, or by the Deepseek and Claude 3.7 Sonnet LLM's has been incorrect in their usage of deprecated calls. To remedy this, I frequently acted as a liaison between the most recently available Qiskit update documentation and Claude 3.7 Sonnet to correct deprecated code and facilitate the smooth implementation of actionable code for submission to the IBM Quantum Platform. While Anthropic's Claude 3.7 Sonnet proved to be indispensable in its assistance with Qiskit, I found the Deepseek LLM to be lacking in its ability to generate any actionable Qiskit code at this time.

For my implementation of FIPS 203, 204, 205 and various other suggested post-quantum cryptographic protocols on my Kali Linux machine, I relied on Open Quantum Safe's Liboqs C library to create actionable benchmark testing of the NIST protocols on my local device. While

the process of downloading the Github repository, importing the associated libraries, and setting up a file system for my testing procedures was not a difficult task, I initially struggled immensely with dependency issues with my global install of Liboqs. Reliance on both documentation from Open Quantum Safe and consultations with the DeepSeek LLM provided no real headway, and I found myself going around in circles for a two-week period of reverting to snapshots within my Kali Linux machine and trying to correctly re-install Liboqs.

Finally, I reached a breakthrough point with a local installation of Liboqs, a correct reconfiguration of my virtual environment, a restructuring of my library installation path, and a new instantiation of Kali Linux. This streamline approach allowed me to continue with my implementation of the local benchmark testing; within which Claude 3.7 Sonnet was again absolutely indispensable in its generation of the base code structure for the benchmark tests, which I was required to edit to solve for deprecation.

Implementing Encryption Infrastructure Management: The Usage of AI.

The scope of this particular research project involved an exploration of the current and publicly available quantum computing platforms, benchmark testing of post-quantum cryptographic protocols recommended by the National Institute of Standards and Technology, and an in-depth research component involving both topics. I have used two large language artificial intelligence models throughout the course of this research project, Anthropic's Claude 3.7 Sonnet LLM and DeepSeek's DeepSeek Chat V3 LLM.

Claude 3.7 Sonnet was essential in its assistance with code generation, management, and debugging capabilities. Without my utilization of the Claude 3.7 Sonnet model, I would not have been able to cover such a large amount of ground with my research and benchmark testing. Within the exploration of both the Qiskit SDK and the benchmark testing for Liboqs, Claude 3.7 Sonnet provided much of the heavy lifting in the code implementation, with myself acting as the guiding force of research and scope of what was being executed and tested. The origination, direction, and testing of the research ideas are my own.

In tandem with my usage of Claude 3.7 Sonnet, I worked with DeepSeek V3 for the organization of research materials. When operating outside of the bounds of the National Institute of standards and Technologies FIPS 203, 204, and 205 primary source documentation, I utilized DeepSeek as a research assistant to provide reference documentation for further exploration, which I personally verified and summarized myself. In addition to this research supplementation, the graphics provided by DeepSeek for various mathematical equations were placed in this paper as a visual expressive component of the research. I was having difficulties in transferring the equations from the cited documentation to the LaTeX format in the Microsoft Word document, so I opted to screenshot and snip the DeepSeek graphic for a cleaner demonstration.

All of the writing within this research project, including captions, research descriptions, process explanations, implementation summaries, infrastructure recommendations, and metaphorical descriptions is/are my own work unless otherwise cited. Any rewriting of summarized material has been cited within the best of my ability.

Implementing Encryption Infrastructure Management: Considerations for further Research.

The scope of this research project has covered a lot of ground from a surface level perspective; my goal is to continue with this research project and to treat it as a living document where I continue to expand upon the various research avenues available to quantum computing and post-quantum cryptographic protocols. My next steps will be to implement benchmark testing for a hybrid M-KEM/AES approach on my Kali Linux device.

Following this, I am considering similar testing on alternative devices in order to measure and report baseline performance metrics for enterprises beginning their cryptographic transitions. A combination of these small scale research approach vectors can provide grassroots support for the larger infrastructural migration to post-quantum cryptographic protocols.

I will continue to involve large language models such as Claude 3.7 Sonnet and DeepSeek V3 within my research and document the progress I make with various models. This information can be used to educate others on the effecting leveraging of artificial intelligence within their own cryptographic infrastructure.

From an academic standpoint, I would also like to continue my research on the foundational aspects of quantum computing, its current and future implementations, and the involvement of quantum computing in all aspects of cryptographic practices, including offensive, defensive, and neutral cybersecurity perspectives. Hopefully, I will continue this work on implementing post-quantum cryptographic algorithms while pursuing my Masters degree in computer science at the University of North Georgia.

References

- Aggarwal, D., Brennen, G. K., Lee, T., Santha, M., & Tomamichel, M. (2017). Quantum attacks on Bitcoin, and how to protect against them. *Ledger*, *3*, 1–12. <https://doi.org/10.5195/ledger.2018.127>
- Amerongen, M. Van. (2021, June 3). *Quantum Technologies in Defence & Security*. NATO Review. <https://www.nato.int/docu/review/articles/2021/06/03/quantum-technologies-in-defence-security/index.html>
- Anthropic. (2025, April 12). [Response to query about executing the Deutsch-Jozsa Algorithm on IBM Quantum Hardware]. Claude 3.7 Sonnet. Retrieved April 12, 2025.
- Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., ... & Stehlé, D. (2021). *CRYSTALS-Kyber – Submission to round 3 of the NIST post-quantum project* (Version 3.02). National Institute of Standards and Technology. <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>
- Babu, H. H. (2023). *Quantum Computing (second edition): A Pathway to Quantum Logic Design*. Institute of Physics Publishing.
- Barrett, P. (2025, January 27). *Useful Quantum Computing is Inevitable-and Increasingly Imminent*. MIT Technology Review. <https://www.technologyreview.com/2025/01/27/1110540/useful-quantum-computing-is-inevitable-and-increasingly-imminent/>
- Bell, J. S. (1964). On the Einstein Podolsky Rosen paradox. *Physics Physique Физика*, 1(3), 195–200. <https://doi.org/10.1103/PhysicsPhysiqueFizika.1.195>
- Bernstein, D. J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., & Schwabe, P. (2019). *The SPHINCS+ signature framework*. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2129–2146). ACM. DOI: [10.1145/3319535.3363229](https://doi.org/10.1145/3319535.3363229)
- Bernstein, D. J., & Lange, T. (2017). *Post-quantum cryptography*. *Nature*, *549*(7671), 188–194. DOI: [10.1038/nature23461](https://doi.org/10.1038/nature23461)
- Birkhoff, G. (1940). *Lattice theory* (Vol. 25). American Mathematical Society.
- Buchmann, J., Dahmen, E., & Hülsing, A. (2011). *XMSS: A practical forward secure signature scheme based on minimal security assumptions*. In *Post-Quantum Cryptography* (pp. 117–129). Springer. DOI: [10.1007/978-3-642-25405-5_8](https://doi.org/10.1007/978-3-642-25405-5_8)

- Callan, T. (n.d.). *NIST's official 2024 Post-Quantum Algorithms*. Sectigo® Official. <https://www.sectigo.com/resource-library/who-are-nists-post-quantum-algorithm-winners>
- Clauser, J. F., Horne, M. A., Shimony, A., & Holt, R. A. (1969). Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15), 880–884. <https://doi.org/10.1103/PhysRevLett.23.880>
- DeepSeek. (2024). *DeepSeek Chat (Version DeepSeek-V3)* [Large language model]. <https://www.deepseek.com>
- Demicheli, G., Martínez, C., & Nascimento, A. C. A. (2023). *Reductions from module lattices to free module lattices*. Cryptology ePrint Archive, Article 886. <https://eprint.iacr.org/2023/886>
- Deutsch, D., & Jozsa, R. (1992). Rapid solutions of problems by quantum computation. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 439(1907), 553–558. <https://doi.org/10.1098/rspa.1992.0167>
- Einstein, A., & Born, M. (1971). *The Born-Einstein letters: Correspondence between Albert Einstein and Max and Hedwig Born from 1916 to 1955* (I. Born, Trans.). Walker & Co. (Original work published 1926).
- Emerging Technology from the arXiv. (2024, August 22). *How Quantum Terrorists could bring down the Future Internet*. MIT Technology Review. <https://www.technologyreview.com/2019/02/08/1388/how-quantum-terrorists-could-bring-down-the-future-internet/>
- Freidin, E. (2025). *Quantum Game Changer*. Lowy Institute. <https://www.lowyinstitute.org/the-interpreter/quantum-game-changer>
- Gómez-Ruiz, F. J., Rodríguez, F. J., Quiroga, L., & Johnson, N. F. (2024, April 11). *Chapter: Vulnerability of Quantum Information Systems to Collective Manipulation*. arXiv.org. <https://arxiv.org/abs/1901.08873>
- IBM (n.d.). *Introduction to Qiskit*. IBM Quantum Documentation <https://docs.quantum.ibm.com/guides>
- IBM. (n.d.). *Install Qiskit*. IBM Quantum Documentation. <https://quantum.cloud.ibm.com/docs/en/guides/install-qiskit>
- Ivezic, M. (2025, February 20). *Quantum Hacking: Cybersecurity of Quantum Systems*. PostQuantum. <https://postquantum.com/post-quantum/quantum-hacking/>
- Katwala, A. (2025, March 24). *The Quantum Apocalypse is Coming. Be Very Afraid*. Wired. <https://www.wired.com/story/q-day-apocalypse-quantum-computers-encryption/>

Laseron, Andrew. (2018). Nuclear Terrorism. SNOVA.

Lyubashevsky, V. (2025, February 14). *Basic lattice cryptography: The concepts behind Kyber (ML-Kem) and Dilithium (ML-DSA)*. IACR Cryptology ePrint Archive. <https://eprint.iacr.org/2024/1287>

Microsoft. Azure Quantum. (n.d.). <https://quantum.microsoft.com/en-us/insights/education/concepts/types-of-qubits>

Mosca, M., & Piani, M. (2025, February 14). *Quantum Threat Timeline Report 2024*. Global Risk Institute. <https://globalriskinstitute.org/publication/2024-quantum-threat-timeline-report/>

Machkovech, S. (2022, September 27). *Intel: "Moore's Law is not Dead" as ARC A770 GPU is priced at \$329*. Ars Technica .Retrieved January 28, 2025 from <https://arstechnica.com/gadgets/2022/09/the-intel-arc-a770-gpu-launches-october-12-for-329/>

Massimo, J., Kampanakis, P., Turner, S., & Westerbaan, B. (n.d.). *Internet X.509 public key infrastructure: Algorithm identifiers for ML-DSA*. IETF Datatracker. <https://datatracker.ietf.org/doc/draft-ietf-lamps-dilithium-certificates/>

National Institute of Standards and Technology, Gallagher, M. P., & Rowe, B., Planning Report 05-2 IPv6 Economic Impact Assessment 42 (2005). Research Triangle Park, NC; RTI International. Retrieved January 28, 2025, from <https://www.nist.gov/system/files/documents/director/planning/report05-2.pdf>

National Institute of Standards and Technology. (2024, August 26). *NIST Releases First 3 Finalized Post-Quantum Encryption Standards*. NIST. (2024, August 26). <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

National Institute of Standards and Technology (2024) Module-Lattice-Based Digital Signature Standard. (Department of Commerce, Washington, D.C.), Federal Information Processing Standards Publication (FIPS) NIST FIPS 204. <https://doi.org/10.6028/NIST.FIPS.204>

National Institute of Standards and Technology. (2024) Module-Lattice-Based Key Encapsulation Mechanism Standard. (Department of Commerce, Washington, D.C.), Federal Information Processing Standards Publication (FIPS) NIST FIPS 203. <https://doi.org/10.6028/NIST.FIPS.203>

National Institute of Standards and Technology (2024) Stateless Hash-Based Digital Signature Standard. (Department of Commerce, Washington, D.C.), Federal Information Processing Standards Publication (FIPS) NIST FIPS 205. <https://doi.org/10.6028/NIST.FIPS.205>

- National Institute of Standards and Technology. (n.d.). *Migration To Post-Quantum Cryptography*. [www.nccoe.nist.gov. https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms](https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms)
- National Institute of Standards and Technology. (2025, March 24). *Quantum Computing Explained*. <https://www.nist.gov/quantum-information-science/quantum-computing-explained>
- Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th ed., pp. 25–28). Cambridge University Press.
- Ounsworth, M., Gray, J., Pala, M., Klaussner, J., & Fluhrer, S. (n.d.). Composite ML-Kem for use in the Internet X.509 Public Key Infrastructure and CMS. <https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-kem-03.html>
- Open Quantum Safe. (2023). *liboqs: C library for quantum-resistant cryptographic algorithms* (Version 0.8.0) [Computer software]. GitHub. <https://github.com/open-quantum-safe/liboqs>
- Press, R. (2025, April 14). *Post-quantum cryptography (PQC): New algorithms for a new era*. <https://www.rambus.com/blogs/post-quantum-cryptography-pqc-new-algorithms-for-a-new-era/>
- P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," Proceedings 35th Annual Symposium on Foundations of Computer Science, Santa Fe, NM, USA, 1994, pp. 124-134, doi: 10.1109/SFCS.1994.365700
- Qiskit contributors. (2023). *Shor's algorithm*. Qiskit Textbook. <https://qiskit.org/textbook/ch-algorithms/shor.html>
- Qiskit contributors. (2023). *Deutsch-Jozsa algorithm*. Qiskit Textbook. <https://qiskit.org/textbook/ch-algorithms/deutsch-jozsa.html>
- Rallapalli, Murthy Ph. D. (2020, February 7). *Implementation of the Deutsch–Jozsa algorithm on IBM Quantum Computer*. LinkedIn. <https://www.linkedin.com/pulse/implementation-deutschjozsa-algorithm-ibm-quantum-murthy>
- Regev, O. (2005) On lattices, learning with errors, random linear codes, and cryptography. Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing STOC '05 (Association for Computing Machinery, New York, NY, USA), pp 84—93. <https://doi.org/10.1145/1060590.1060603>.
- Schwabe, P. (2020.). *CRYSTALS Cryptographic Suite for Algebraic Lattices*. [pq-crystals.org. https://pq-crystals.org/dilithium/index.shtml](https://pq-crystals.org/dilithium/index.shtml)

Shor, P. W. (1997). *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*. SIAM Journal on Computing, 26(5), 1484–1509.
<https://doi.org/10.1137/S0097539795293172>

Text - H.R.6227 - 115th Congress (2017-2018): National Quantum Initiative Act. (2018, December 21). <https://www.congress.gov/bill/115th-congress/house-bill/6227/text>

Zhu, L. (Analyst in telecommunications policy) (2023). *Quantum Computing: Concepts, Current State, and Considerations for Congress*. Congressional Research Service.